

SynthOmnicon Framework: Comprehensive Usage Guide (v0.4.0)

March 20, 2026

Version 0.4.0 — Quantum primitive extensions: T_{braid} · K_{MBL} · $\Gamma_{\text{(DISSIPATIVE)}}$ · QUANTUM grammar tier · Ω (**TopoIndex**) — **11th primitive** · Factor 8 (quantum criticality) · `syncon distance` command · all algebra commands wired to `syncon` CLI.

Version 0.3.8 — Quantum domain encoding: 5 quantum particles (photon/proton/electron/spin/qubit) · G_{aleph} first appearance · Axiom 1 as classical boundary detector · spin singlet $F_{\text{ell}} \rightarrow F_{\text{hbar}}$ correction.

Version 0.3.7 — Ice polymorph catalog (13 phases) · $T_{\in}$ sub-label integration across constraints/algebra/ensampler/perturbation/cli · domain-agnostic agent prompts · `syncon compare` all 10 primitives · Key Justifications fix.

Version 0.3.6 — $T_{\cup}$ bowl topology (8th T value, found via catalog self-audit) · $T_{\in}$ sub-labels: $T_{\in}(\text{hex})$, $T_{\in}(\text{mixed})$, $T_{\in}(\times 2)$, $T_{\in}(\text{sym})$ (12 total T values) · 222 bowl reclassifications · Solid-angle I_{angle} formula · Axiom 4 scan · Junk purge case-insensitive fix.

Version 0.3.5 — Phase 3a DSL · `syn` YAML runner · `syncon run` · Agent fixes (ten primitives) · Topology symbols $T_{\perp}$, $T_{\perp}$, $T_{\in}$.

Version 0.3.3 — Experimental Validation Suite: CB[7] 6/6 HotSwap validation · F_{ℓ} tier activation · Factor 7 (Frank-model bifurcation) · Soai catalog entry · Proline-aldol Varma probe.

Version 0.3.2 — Tuple Algebra + Compositional Design Language: `meet`, `join`, `path`, `tensor`, `lift`, `pipeline`.

Version 0.3.0 — Four new analysis protocol modules: SYNTHONIC_PERTURBATION, SYNTHONIC_TRAJECTORY, SYNTHONIC_ENSEMBLER, SYNTHONIC_RETRODESIGN.

Full Notation: ; T; R; P; F; K; G; Γ ; Φ ; S; Ω (*Ω optional, defaults None for classical synthons*)

0.1 Introduction to SynthOmnicon

0.1.1 Foundational Insight: Primitives as Directed Constraints

The eleven primitives ; T; R; P; F; K; G; Γ ; Φ ; S; Ω are **relational operators**, not intrinsic — — *none describes an isolated property of a system standing alone.*

This has a practical consequence: the framework’s confirmed predictions (see PRIMITIVE_PREDICTIONS.md) were all derived from **ordinal comparisons**, not absolute values. The CB[7] displacement hierarchy (6/6 experimental matches) was predicted from $F_h > F_{eth} > F_\ell$ alone. The Soai Frank-bifurcation was predicted from the co-occurrence pattern $D_{inf} + T_+ + P_{DA} + F_h$. Intrinsic scalar properties -- binding enthalpy, hydrophobicity, gap magnitude -- were not required as inputs.

The compositional algebra (meet, join, tensor, path, lift) has no unary information generators. Every operation requires at least one additional operand -- environment, partner, or target state. You cannot call `tensor(photon)` without a second argument; the algebra returns an error. A synthon’s tuple describes its *interaction-ready potential*; the algebra computes only when that potential is actualized against another term.

0.1.2 What’s New in v0.4.1?

Molecular Domain Catalog -- Persistent Synthon Registration (March 17, 2026):

1. `synthomnicon/domains/molecular/__init__.py` -- `register_molecular_synthons()` added. Nine canonical molecular and supramolecular synthons are now programmatically registered at import time, surviving catalog JSON resets:
2. **Design suite:** all 20 `.syn` scripts execute without [ERROR] -- 18 succeed, 2 are intentional F-floor pedagogical demonstrations (designs 01 and 04). All previously [ERROR] failures caused by missing catalog entries are now resolved.
3. `register_molecular_synthons()` wired to `synthomnicon/__init__.py` -- auto-invoked on import `synthomnicon`, idempotent. Registration order: cross-domain $\rightarrow$ quantum $\rightarrow$ molecular.

0.1.3 What’s New in v0.3.0?

Four Protocol Modules (March 15, 2026):

1. **SYNTHONIC_PERTURBATION** (`synthomnicon/perturbation.py`):
 - `PerturbationEngine.sweep_all(synthon, delta_g)` -- primitive Jacobian: $\Delta\xi_{CP}$ for every primitive ± 1 tier
 - `PerturbationEngine.fault_injection(synthon, delta_g)` -- single-point-of-failure analysis
 - `PerturbationEngine.find_path_to_target(synthon, delta_g, target_xi_CP, optimize_primitives)` -- minimum-step tuning path
 - CLI: `syncon perturb sweep <name> -delta-g <float>`
2. **SYNTHONIC_TRAJECTORY** (`synthomnicon/trajectory.py`):
 - `TemporalSynthonAgent` -- encode D_∞ systems as step sequences, validate Axiom 6 compliance
 - Three checks: S mass balance, Axiom 4 (D_∞ or R_-), $K_{trap}/\Delta G > 100$
 - CLI: `syncon trajectory validate -steps <names> -reset <name>`
3. **SYNTHONIC_ENSEMBLER** (`synthomnicon/ensembler.py`):
 - `EnsembleCatalog` -- $N \times N$ pairwise compatibility, emergent property detection, system ξ_{CP}

Synthon	D	T	F	Role
nitroso_radical	D_∞	T_	F_h	Temporal autocatalytic redox cycle; Frank-model Factor 7 fires (score > 0.3); start point for criticality-ascent designs
amide_dimer	D_∧	T_	F_eth	N-H...O=C H-bonded dimer; F_eth weaker than carboxylic acid dimer (F_h); lattice floor/ceiling pair for design 03
nitroso_radical	D_	T_cage	F_h	Deep-cavity anion- π cavitand; shape-selective cage, K_slow
nitroso_radical	D_	T_bowl	F_h	Calixarene bowl; open cup, T_bowl < T_cage in ordinal (fallback partner in or-strategy)
nitroso_radical	D_	T_cage	F_eth	Crown ether host-guest; flexible macrocycle $\rightarrow$ F_eth (fidelity bottleneck partner in design 12)
nitroso_radical	D_	T_cage	F_h	Cryptand cage; 3D bicyclic preorganisation $\rightarrow$ F_h (design 16 start)
nitroso_radical	D_	T_cage	F_h	CB[n] barrel; high rigidity (F_h), slow dethreading (K_slow); tensor partner in designs 12 and 16
3				
synthon_methyl_	D_∧	T_	F_eth	CH carbanion; P_minus; retrodesign

- Emergent: criticality, $G \rightarrow G$ amplification (Axiom 3), interface fidelity degradation
 - CLI: `syncon ensemble check -components <names>`
4. **SYNTHONIC_RETRODESIGN** (`synthomnicon/retrodesign.py`):
- `RetrodesignEngine.decompose()` -- recursive axiom-pruned decomposition tree
 - Axioms 1, 2 (sub-tuples claiming G only), 4, 6
 - CLI: `syncon retrodesign <name_or_notation> -max-depth 3 -prune-axioms 1,2,4,6`

See **Section 7** for full protocol API documentation and CLI reference. See **Section 8** for the tuple algebra commands.

0.1.4 What's New in v0.3.5?

Phase 3a DSL + Agent Fixes + Topology Symbols (March 16, 2026):

1. **.syn YAML DSL** (`synthomnicon/syn_runner.py`): Design programs compiled to SynthonM pipelines. Supported step types: `join`, `meet`, `tensor`, `lift`, `path`, `assert`, `bind`, `or`. The `or` step implements MonadPlus fallback (`strategy_or`). Post-hoc output: `assert`: block checks WriterT-level cost (`total_delta_xi < N`, `steps <= N`, `criticality_ok`).
2. **syncon run <file.syn>**: New CLI command. `-format text|json`, `-save PATH`, `-dry-run`. Runs .syn scripts and reports step-by-step trace plus output assertions.
3. **Agent primitive awareness**: Both `axiom_guided_generator` and `synthon_generator_ag` now declare **ten primitives** (D , T , R , P , F , K , G , Γ , Φ , S). `criticality_phase` parser defaults to `Phi_sub` instead of returning `None`. Topology block lists all 7 values with symbols.
4. **New topology symbols**: Three acyclic connectivity classes now have explicit symbols:
 - $T_{\mid}$ (LINEAR) -- unbranched linear chain, no junction nodes
 - $T_{\perp}$ (BRANCHED) -- branched acyclic, one or more junction nodes
 - T_{in} (NETWORK) -- multiply-connected, cycles permitted without cage closure All three are registered in `Topology.from_symbol()`.
5. **Notation always includes Φ** : `to_notation()` defaults to `Phi_sub` when `criticality` is unset; `S` appended as 10th position when set. Notation is always a 9- or 10-element tuple.

0.1.5 What's New in v0.3.3?

Experimental Validation Suite + Factor 7 + F_{ℓ} Activation (March 16, 2026):

1. **CB[7] competitive displacement -- 6/6 HotSwap validation**: Three-tier CB[7] series (Fc/Ad/DABCO) tests the F-floor asymmetric ratchet against Kim JACS 2001 / Assaf & Nau CSR 2015 experimental data. All 6 directional predictions (3 APPROVED, 3 BLOCKED) match experiment from ordinal F ranking alone. First use of F_{ℓ} tier (DABCO, $K_a = 2 \times 10 \text{ M}\ddot{z} < \text{threshold } \sim 10 \text{ M}\ddot{z}$).
2. **Factor 7 -- Frank-model classical bifurcation** (`varma_probe.py`): New heuristic factor (weight 0.25) in `score_phi_c_candidacy()`. Fires when

$D_\infty + T_- + P_{\text{directional}} + F_h$ are co-present. Identifies pitchfork bifurcation at $ee = 0$ (Frank 1953) -- universality class distinct from Varma QXY and steric-cliff.

3. **Soai reaction catalog entry** (soai_pyrimidyl_autocatalytic_cycle): Fully grounded. $\xi_r = 15$, $i_\tau = 7.2 \times 10^2$ ($\omega_c = 10^2 \text{ s}^2$), ratio = 0.94. Probe score 0.920 (approaching Φ_c , Frank-model mechanism). Highest-confidence Φ_c candidate in catalog.
4. **Proline-aldol Varma probe**: $\xi_r = 6.2$, $i_\tau = 1.8 \times 10^2$, ratio = 0.189 $\rightarrow \Phi_{\text{sub}}$ confirmed. The 60 Å correlation length required for criticality is incompatible with the observed enamine geometry.
5. **Three-mechanism discrimination** -- probe now discriminates three universality classes:
 - Varma QXY (Factors 1-5)
 - Steric-cliff proxy (Factor 6, db24c8 score 0.461)
 - Frank-model classical bifurcation (Factor 7, Soai score 0.920)

0.1.6 What's New in v0.3.2?

Tuple Algebra + Compositional Design Language (March 16, 2026):

1. **Lattice operations** -- syncon meet S1 S2 and syncon join S1 S2. Componentwise min/max on ordered primitives; CONFLICT sentinel on categorical mismatch; Φ_c dominates in both.
2. **Path search** -- syncon path SRC DST. BFS over valid-swap directed graph restricted to same {D,T} cluster; accumulates $\Delta\xi_{\text{CP}}$; correctly asymmetric (F-floor enforcement).
3. **Tensor product** -- syncon tensor S1 S2. Ensemble prediction: $F \rightarrow \min$ (bottleneck $K \rightarrow \min$, $G \rightarrow \max$, Φ_c propagates, $\xi_{\text{ens}} = \xi + \xi - \lambda \cdot I(s;s)$).
4. **Natural transformations** -- syncon lift NAME (temporal|spatial|critical|molecular). Four primitive-rewriting maps with side-by-side diff output. Criticality lift gated on $F \geq F_h$ (Axiom 5 enforcement).
5. **DesignPipeline** -- syncon pipeline START -step op:arg [...]. Writer+Maybe monad: chained operations with automatic ξ_{CP} threading and fail-fast logging. Equivalent to Haskell do-notation over the HotSwap category.

See Section 8 for full reference.

0.1.7 What's New in v2.2?

I(bits) Calibration + Quantitative Criticality + Stoichiometry (March 2026):

1. **Calibrated I(bits) pipeline** -- DOF-counting replaces 4-6 bit heuristic (new range: 6-18 bits). syncon info-bits -calibrate [-solvent chloroform|THF|DMSO].
2. **Quantitative criticality probe** -- z_{eff} divergence measure + degeneracy strength score (0-1, four tiers). syncon criticality-probe -batch -degeneracy-type -export-candidates.
3. **Stoichiometry primitive S** -- 10th primitive with weight 0.08 in similarity scoring. syncon catalog auto-stoichiometry backfills T_- entries. Pass 4 audit enforces S consistency.

4. **Analogy enhancements** -- syncon analogies -critical-only and -stoichiometry-aware flags.
5. **Fidelity tier boundaries updated** -- HIGH $\leq$ 8.5 nats · MEDIUM 8.5–11.0 nats · LOW > 11.0 nats (was 9.0/11.5).

0.1.8 What's New in v2.1?

SYNTHONICON_FIXES.md Full Implementation (March 14–15, 2026):

1. **Grounding Validation with Registration Blocking** (Fix 1 -- CRITICAL):
 - -strict-grounding flag blocks registration if primitives lack mechanistic grounding
 - -override-grounding with -override-reason for speculative entries
 - -speculative flag registers into isolated 'speculative' domain
 - Audit trail logs all grounding overrides with human-provided justifications
 - Grounding metadata persisted in catalog JSON
2. **Axiom 6: Temporal Grounding** (Fix 2 -- HIGH, extended v0.3.3):
 - D_{∞} requires either a named discrete reset OR a continuously supplied dissipative flux (see below)
 - **Discrete reset** (reset_type="discrete"): validates cycle_steps $\geq$ 2, or axiom6_grounding block with initial_state, transformation, work_perform, reset_mechanism
 - **Continuous dissipative** (reset_type="continuous"): validates driving_gradient and driving_gradient.coupling in synthon_grounding["reset"]
 - Keyword scan (AXIOM_6_RESET_INDICATORS, AXIOM_6_PROCESS_INDICATORS) is fallback only when no structured block is present
3. **Axiom 7: Cyclic Topology Grounding** (Fix 3 -- HIGH):
 - T_{now} requires named closing bond/interaction
 - Detects invalid justifications (linear, rod, chain, axial, etc.)
 - Keyword indicators: AXIOM_7_CLOSING_INDICATORS, AXIOM_7_INVALID_TOPO_KEYWORDS
4. **Per-Primitive Confidence** (Fix 4 -- MEDIUM):
 - PrimitiveGrounding dataclass: confidence: float, is_grounded: bool, failure_reason, suggested_alternative
 - ADVERSARIAL_GROUNDING_PROMPT -- challenges each primitive from first principles
 - Confidence auto-derived from status (GROUNDED=0.9, AMBIGUOUS=0.5, UNGROUNDED=0.1, INVALID=0.0)
5. **Quantum Extension Quarantine** (Fix 5 -- MEDIUM):
 - domain field tags speculative/quantum entries
 - Prevents semantic contamination of grounded catalog
6. **Catalog Audit Command** (Fix 6 -- MEDIUM):
 - syncon audit scans catalog for grounding issues
 - Axiom 6/7 shortcuts, -auto-flag, -dry-run support (see Section 4.2)
7. **NLP Format Enforcement** (v2.1.3):
 - All LLM prompts across the codebase follow NLP_FORMAT.md
 - XML tags, **MUST**/**MUST NOT**, declarative commands, explicit output

formats

Eight Axioms Now Enforced:

- Axioms 1-5: Composition axioms (QUANTSYNTHONICON.md Section IV)
- Axioms 6-7: Grounding axioms (SYNTHONICON_FIXES.md)
- Axiom 8: R physics match (R must match actual interaction physics)

SynthOmnicon is a groundbreaking Python framework that brings the theoretical elegance of the **Unified Synthonicon** to practical application. At its core, SynthOmnicon provides a computational platform for systematically analyzing, designing, and predicting the behavior of self-organizing chemical systems. It unifies disparate fields--from traditional molecular organic chemistry to complex supramolecular assemblies and dynamic temporal processes--under a single, coherent conceptual umbrella.

The framework's primary goal is to bridge the understanding gap between these domains by employing a shared language: the **Ten Primitives** (Dimensionality, Topology, Recognition Mode, Polarity, Fidelity, Kinetic Character, Granularity, Interaction Grammar, Criticality Phase, and Stoichiometry). This unified notation allows researchers and developers to describe any self-organizing chemical motif, regardless of its scale or mechanism, with unprecedented precision.

Key Benefits of SynthOmnicon:

- **Unified Language:** Provides a common vocabulary for describing chemical systems across molecular, supramolecular, and temporal scales, fostering interdisciplinary insights.
- **Predictive Power:** Enables the prediction of system behavior, compatibility, and efficiency based on the fundamental properties of its constituent synthons.
- **Axiom Validation:** Automatically validates synthons against five composition axioms from QUANTSYNTHONICON.md Section IV.
- **AI-Driven Design:** The discrete and quantifiable nature of its primitives makes SynthOmnicon an ideal foundation for AI and machine learning applications in chemical synthesis and materials discovery. By encoding chemical knowledge in a structured format, it facilitates generative design and autonomous chemical research.

0.1.9 Core Philosophy: Synthons as Constraint Propagation Units

At the heart of SynthOmnicon lies the concept of a **Synthon** as a "constraint propagation unit." Traditionally, synthons were idealized fragments used in retrosynthetic analysis. SynthOmnicon extends this definition: a Synthon is understood as a minimal subsystem that **encodes a specific set of constraints**. When this synthon interacts with a compatible environment via a defined recognition mode, it actively *reduces the system's degrees of freedom*. This reduction reliably steers the overall system towards a predictable target state--be it a specific molecular connectivity, a complex supramolecular geometry, or a precise temporal sequence.

This information-theoretic perspective positions synthons not merely as static building blocks, but as dynamic agents that guide assembly and transformation by influencing the probabilistic landscape of chemical outcomes. In essence, synthons act as local rules that propagate their influence globally, analogous to constraint propagation algorithms in computer science, where local rules reduce a vast search space to a manageable set of solutions.

0.1.10 The Ten Primitives: A Unified Language

The Unified Synthonicon is built upon ten primitives, each representing a crucial aspect of a synthon's behavior and function. These primitives form a standardized, extensible vocabulary:

Together, these primitives are used to generate the full **Unified Notation**:

`; T; R; P; F; K; G; Γ; Phi; S`. *This notation is the cornerstone of SynthOmnicon, enabling quantitative domain comparisons.*

0.2 Installation and Setup

To begin using the SynthOmnicon framework, follow these steps to set up your Python environment and install the necessary dependencies.

0.2.1 Prerequisites

- **Python Version:** SynthOmnicon requires Python 3.9 or higher. To check your Python version, run: `python -version` or `python3 -version`.
– If you need to install Python, visit python.org.
- **Operating System:** The framework is developed and tested on Linux environments, but should generally work on macOS and Windows (using WSL for a better experience on Windows).
- **Internet Connection:** Required for downloading dependencies.

0.2.2 Recommended Environment Management

It is highly recommended to use a virtual environment to manage project dependencies. This isolates the project's packages from your system-wide Python installation, preventing conflicts and ensuring reproducibility. Two excellent tools for this are `uv` (faster, modern alternative) or `venv` (standard Python module).

0.2.3 Installation Steps

Option 1: Using `uv` (Recommended for Speed and Efficiency) `uv` is a new, highly performant Python package installer and resolver. If you don't have `uv` installed, you can get it with `pip`:

```
pip install uv
```

Once `uv` is installed, navigate to your SynthOmnicon project directory and follow these commands:

1. Create a virtual environment:

```
uv venv
```


Primitive	Symbol	Description	Key Purpose in SynthOmnicon
Dimensionality	$D (D_{\wedge}, D_{\Delta}, D_{\infty})$	The coordinate set along which the synthon operates.	Defines the operational scale (molecular, supramolecular, temporal).
Topology	$T (\$T_{\backslash\text{bowtie}}, T_{\backslash\text{ggg}}, T_{\backslash\text{square}}, T_{\backslash\text{square}\backslash\text{square}}, T_{\backslash}$	$\}, T_{\backslash\text{perp}}, T_{\backslash\text{in}}\$)$	The internal connectivity pattern of the synthon's minimal motif.
Recognition Mode	$R (R_{\subseteq}, R_{\supseteq}, R_{\dagger}, R_{\Leftrightarrow})$	The physical mechanism of interaction.	Specifies how synthons interact (covalent, non-covalent, dynamic).
Polarity	$P (P+, P-, P_{\pm})$	The directional character of the interaction.	Dictates partner preference and orientation (acceptor, donor, self-complementary).
Fidelity	$F (F_h, F_{\partial}, F_{\ell})$	Thermodynamic reliability anchored to ξ_{CP} . Tiers: HIGH ≤ 8.5 nats · MEDIUM 8.5–11.0 nats · LOW > 11.0 nats.	Quantifies the predictability and robustness of an interaction.
Kinetic Character	$K (K_{\text{fast}}, K_{\text{mod}}, K_{\text{slow}}, K_{\text{tr}})$	Activation barrier and pathway multiplicity.	Distinguishes thermodynamic fidelity from operational accessibility.
Granularity	$G (G_{\sqsubset}, G_{\sqsupset}, G_{\mathbb{N}})$	The scale of control exerted by the synthon.	Defines the scope of influence (local, mesoscale, global).
Interaction Grammar	$\Gamma (\Gamma_{\wedge}, \Gamma_{\vee}, \Gamma_{\rightarrow}) \times (\text{SPECIFIC} \cdot \text{SELECTIVE} \cdot \text{BROAD})$	The logic governing partner selection.	Determines the specificity and ordering of binding.
Criticality Phase	$\Phi (\Phi_{\text{sub}}, \Phi_c, \Phi_{\text{super}})$	Phase relative to the G–D criticality locus.	Encodes whether the synthon exhibits scale-free behavior (Axiom 5).
Stoichiometry	S (e.g. "1:1", "2:1", "n:m")	Valency ratio of the recognition event; constrains T and P. Weight 0.08 in analogy scoring.	Distinguishes homodimers from host–guest and polymeric assemblies.

This creates a new virtual environment named `.venv` in your project root.

2. Activate the virtual environment:

- `source .venv/bin/activate`

On Linux/macOS:

- `source .venv/bin/activate`

On Windows (Git Bash/WSL):

- `.venv\Scripts\activate.bat`

On Windows (Command Prompt):

- `.venv\Scripts\Activate.ps1`

On Windows (PowerShell):

Your command prompt should now show `(.venv)` prefixed, indicating the virtual environment is active.

3. Install project dependencies:

```
uv pip install -r requirements.txt
```

`uv` will quickly install all packages listed in `requirements.txt`.

Option 2: Using `pip` and `venv` (Standard Python) If you prefer to use the standard Python tools, `venv` and `pip` are readily available.

1. Create a virtual environment:

```
python -m venv .venv
```

This creates a new virtual environment named `.venv` in your project root.

2. Activate the virtual environment:

- `source .venv/bin/activate`

On Linux/macOS:

- `source .venv/bin/activate`

On Windows (Git Bash/WSL):

- `.venv\Scripts\activate.bat`

On Windows (Command Prompt):

- `.venv\Scripts\Activate.ps1`

```
.venv\Scripts\Activate.ps1
```

On Windows (PowerShell):

Your command prompt should now show (.venv) prefixed, indicating the virtual environment is active.

3. Install project dependencies:

```
pip install -r requirements.txt
```

pip will install all packages listed in requirements.txt.

0.2.4 Verifying Installation

After installing dependencies, you can run the integration tests to verify that everything is set up correctly:

```
python test_integration.py
```

You should see a message indicating that all tests passed. If any tests fail, please refer to the "Troubleshooting" section or create an issue in the project repository.

0.2.5 Project Structure Overview

The SynthOmnicon framework is organized into several key directories:

- `synthomnicon/`: Contains the core Python modules for the Unified Synthonicon, including primitive definitions, the Synthon dataclass, catalog, constraints engine, and thermodynamics calculations.
- `synthomnicon/domains/`: Houses domain-specific agents (molecular, supramolecular temporal, hybrid) that leverage the core framework for specialized analyses.
- `framework/`: Includes components of the underlying AjintK multi-agent framework, providing utilities for agent orchestration and communication.
- `examples/`: Demonstrative scripts showing how to use various parts of SynthOmnicon.
- `QUANTSYNTHONICON.md`: The foundational theoretical document detailing the Unified Synthonicon.
- `README.md`: General project information and quick start guide.
- `requirements.txt`: Lists all Python dependencies.
- `test_integration.py`: The suite of integration tests.

0.3 Core Concepts: The Ten Primitives in Detail

The Unified Synthonicon framework categorizes the fundamental properties of chemical interactions into ten primitives. Primitives 1-9 are encoded as Python Enum classes; primitive 10 (S, Stoichiometry) is a string field with category-aware similarity grading and Pass 4 audit enforcement. Together they form the standardized language for describing, comparing, and predicting synthon behavior across molecular, supramolecular, and temporal domains.

0.3.1 Dimensionality (D)

Dimensionality specifies the coordinate set or "space" along which a synthon primarily operates. It helps define the context and scale of the chemical system under consideration. The Dimensionality enum can also represent hybrid systems that span multiple domains.

- **Enum Members:**

- MOLECULAR ("D_wedge" or "D_and"): Point-like reactivity, typically involving individual atoms or small molecules.
- SUPRAMOLECULAR ("D_triangle" or "D_"): Three-dimensional spatial organization, such as crystal packing or host-guest interactions.
- TEMPORAL ("D_infinity" or "D_inf"): One-dimensional periodicity over time, characteristic of oscillating reactions or catalytic cycles.
- HYBRID_MOL_SUPRA ("D_wedge_triangle"): Combines molecular-level reactions within a supramolecular framework.
- HYBRID_MOL_TEMP ("D_wedge_infinity"): Molecular reactions with a temporal component (e.g., dynamic covalent chemistry).
- HYBRID_SUPRA_TEMP ("D_triangle_infinity"): Supramolecular structures exhibiting temporal dynamics.
- HYBRID_ALL ("D_all"): Spans all three fundamental dimensions.

- **Usage Examples:**

```
from synthonicon.models import Dimensionality

# Accessing enum members
mol_dim = Dimensionality.MOLECULAR
print(f"Molecular Dimensionality: {mol_dim.value}")

# Parsing from symbolic notation
temp_dim = Dimensionality.from_symbol("D_inf")
print(f"Temporal Dimensionality from symbol: {temp_dim.name}")

# Hybrid example
hybrid_dim = Dimensionality.HYBRID_MOL_SUPRA
print(f"Hybrid Dimensionality: {hybrid_dim.value}")
```

- **domains Property:** Each Dimensionality member has a domains property, which returns a Set[str] indicating the fundamental domains it encompasses (e.g., "molecular", "supramolecular", "temporal").

```
print(f"Domains for HYBRID_MOL_SUPRA: {hybrid_dim.domains}")
# Expected: {'molecular', 'supramolecular'}
print(f"Domains for TEMPORAL: {temp_dim.domains}")
# Expected: {'temporal'}
```

0.3.2 Topology (T)

Topology describes the internal connectivity pattern or spatial arrangement within the synthon's minimal motif. It characterizes the geometric scaffolding or network structure formed by the interacting components.

- **Enum Members:**

- CYCLIC_BOWTIE ("T_bowtie" or "T_"): Cyclic motifs, like hydrogen-bonded dimers (e.g., carboxylic acid dimer) or catalytic cycles.
- CHAIN ("T_chains" or "T_"): Linear or extended chain structures (e.g., polymers, protein helices).
- HUB_NODE ("T_square" or "T_"): Central nodes connecting multiple branches, common in metal-organic frameworks (MOFs) or dendritic structures.
- LINEAR ("T_linear" / "T_|"): Unbranched linear chain, no junction nodes (e.g., linear polymers, rod-like coordination chains).
- BRANCHED ("T_branched" / "T_⊥"): Branched acyclic topology with one or more junction nodes (e.g., dendrimers, branched polymers).
- * NETWORK ("T_network" / "T_in"): Multiply-connected, cycles permitted but without full 3D cage closure (e.g., MOF nets, 2D coordination networks). Use a sub-label when ring topology is known: NETWORK_HEX ("T_network_hex" / "T_in(hex)": 6-membered rings only (e.g., ice Ih, honeycomb MOFs, graphene-like nets). Complexity 5.
- * NETWORK_MIXED ("T_network_mixed" / "T_in(mixed)": Mixed ring sizes (e.g., ice III, ice IV, ice V). Complexity 5.
- * NETWORK_INTERPENETRATING ("T_network_interp" / "T_in(x2)": Two independent sub-networks occupying the same space (e.g., ice VI, ice VII, ice VIII -- bcc interpenetrating H-bond lattices). Complexity 6.
- * NETWORK_SYM ("T_network_sym" / "T_in(sym)": Centrosymmetric bonding -- symmetric H-bond or equivalent (e.g., ice X at ≥ 70 GPa). Complexity 5.
- CAGE ("T_cage" / "T_"): Fully enclosed, hollow structures -- 3D closure required (e.g., cucurbiturils, Fujita spheres, COCs).
- BOWL ("T_bowl" / "T_⊔"): Open concave cavity with a single portal -- guest enters and exits freely; K_fast is the default (e.g., calixarenes, resorcinarenes, pillar[n]arenes, cyclotrimeratrylenes). Distinguished from T_ by the absence of full 3D closure: $T_{\sqcup} \rightarrow T_-$ is non-conservative (changes kinetic regime). *Identified through catalog self-audit in v0.3.6.*
- BRAID ("T_braid" / "T_"): **NEW in v0.4.0.** Anyonic/braided exchange statistics -- the topology of braid groups, not spatial connectivity. Encodes systems where particle exchange is described by non-abelian braid operations rather than simple permutations (Z for bosons/fermions). $\text{tensor}(T_{\text{braid}}, T_{\text{braid}}) \rightarrow T_{\text{braid}}$ (anyonic statistics preserved). $T_{\text{braid}} \quad T_{\text{linear}} = \perp$ (no classical topology sits below both). Complexity 4. Physical systems: fractional quantum Hall states ($\nu=1/3$, $\nu=5/2$), Kitaev honeycomb B-phase, non-abelian Majorana platforms. *Identified as missing from the T lattice via quantum particle encoding (v0.4.0).*

- Usage Examples:

```
from synthonicon.models import Topology

cyclic_topo = Topology.CYCLIC_BOWTIE
print(f"Cyclic Topology: {cyclic_topo.value}")

# Accessing from symbol
hub_topo = Topology.from_symbol("T_")
print(f"Hub-Node Topology from symbol: {hub_topo.name}")
```

- **complexity Property:** The complexity property returns an integer score (1-5) reflecting the topological intricacy, where higher values indicate more complex arrangements.

```
print(f"Complexity of CYCLIC_BOWTIE: {cyclic_topo.complexity}")
# Expected: 2
print(f"Complexity of NETWORK: {Topology.NETWORK.complexity}")
# Expected: 5
```

0.3.3 Recognition Mode (R)

Recognition Mode defines the specific physical mechanism by which synthons interact and recognize each other, enabling reliable propagation of information or structure.

- Enum Members:

- COVALENT ("R_subset" or " $R_{\subseteq}$ "): Involves the formation of strong, directional covalent bonds.
- NON_COVALENT ("R_superset" or " $R_{\supseteq}$ "): Relies on weaker, reversible interactions like hydrogen bonds, van der Waals forces, or halogen bonds.
- DYNAMIC_CATALYTIC ("R_dagger" or " $R_{\dagger}$ "): Encompasses interactions mediated by dynamic processes, often catalytic, leading to reversible bond formation or transformation.
- MECHANICAL ("R_mechanical" or " R_{mech} "): Characterizes interlocked molecules (e.g., rotaxanes, catenanes) where components are physically linked but not covalently bonded.
- COVALENT_DYNAMIC ("R_covalent_dynamic" or " $R_{\subseteq+}$ "): Hybrid mode for dynamic covalent chemistry, combining covalent bond formation with reversibility.

- Usage Examples:

```
from synthonicon.models import RecognitionMode

non_covalent_mode = RecognitionMode.NON_COVALENT
print(f"Non-Covalent Recognition Mode: {non_covalent_mode}")
```

```

        .value}")

# Accessing from symbol
dynamic_mode = RecognitionMode.from_symbol("R_")
print(f"Dynamic Catalytic Mode from symbol: {dynamic_mode
      .name}")

```

- **Properties:**

- `interaction_energy_range` (Tuple[float, float]): Returns a typical energy range (in kJ/mol) for interactions of this mode.
- `is_reversible` (bool): Indicates whether the interaction is typically reversible.

```

print(f"Energy range for COVALENT: {RecognitionMode.
      COVALENT.interaction_energy_range} kJ/mol")
# Expected: (150.0, 500.0)
print(f"Is NON_COVALENT reversible? {non_covalent_mode.
      is_reversible}")
# Expected: True

```

0.3.4 Polarity (P)

Polarity captures the directional character of the interaction, dictating how components orient themselves and select partners. It relates to electron density distribution and reactivity.

- **Enum Members:**

- `ACCEPTOR` ("P_plus" or "P+"): Electron-deficient site, acts as an electrophile or hydrogen bond acceptor.
- `DONOR` ("P_minus" or "P-"): Electron-rich site, acts as a nucleophile or hydrogen bond donor.
- `SELF_COMPLEMENTARY` ("P_pm" or "P_+/-"): Possesses both donor and acceptor characteristics that allow it to interact with identical motifs (e.g., carboxylic acid dimers).
- `DONOR_ACCEPTOR` ("P_directional" or "P_+-"): Combines donor and acceptor sites in a directional manner, forming specific D-A pairs.

- **Usage Examples:**

```

from synthonnicon.models import Polarity

acceptor_pol = Polarity.ACCEPTOR
print(f"Acceptor Polarity: {acceptor_pol.value}")

# Accessing from symbol
self_comp_pol = Polarity.from_symbol("P_+/-")
print(f"Self-Complementary Polarity from symbol: {
      self_comp_pol.name}")

```

- **Methods:**

- `is_directional (bool)`: Indicates if the polarity type implies inherent directionality.
- `is_compatible_with(other: Polarity) (bool)`: Checks if two polarity types are compatible for interaction.

```
print(f"Is ACCEPTOR directional? {acceptor_pol.
      is_directional}")
# Expected: True
print(f"Is ACCEPTOR compatible with DONOR? {acceptor_pol.
      is_compatible_with(Polarity.DONOR)}")
# Expected: True
print(f"Is ACCEPTOR compatible with SELF_COMPLEMENTARY? {
      acceptor_pol.is_compatible_with(Polarity.
      SELF_COMPLEMENTARY)}")
# Expected: False
```

0.3.5 Fidelity (F)

Fidelity provides a nuanced measure of a synthon's reliability, predictability, and robustness. It quantifies how consistently a synthon will perform its intended function under given conditions. Fidelity is domain-dependent and can be quantified differently (e.g., bond dissociation energy, interaction energy, fidelity per cycle).

- **Enum Members:**

- HIGH (" F_{hbar} " or " F_h "): Dominant, highly reliable, geometry-enforcing interactions. $\xi_{\text{CP}} \leq 8.5$ nats. In supramolecular host-guest systems: $K_a \sim 10$ M $\ddot{\text{z}}$ (e.g., CB[7]·ferrocene-ammonium, $K_a \approx 3 \times 10^2$ M $\ddot{\text{z}}$). Represents strong constraint.
- MEDIUM (" F_{eth} " or " F_- "): Context-dependent, robust but conditional interactions. $\xi_{\text{CP}} 8.5\text{--}11.0$ nats. $K_a \sim 10\text{--}10$ M $\ddot{\text{z}}$ (e.g., CB[7]·adamantane-ammonium, $K_a \approx 4 \times 10$ M $\ddot{\text{z}}$).
- LOW (" F_{ell} " or " F_ℓ "): Probabilistic, competition-sensitive interactions, less reliable. $\xi_{\text{CP}} > 11.0$ nats. $K_a \sim 10$ M $\ddot{\text{z}}$ (e.g., CB[7]·DABCO, $K_a \approx 2 \times 10$ M $\ddot{\text{z}}$). The F_ℓ tier boundary is anchored by the CB[7] competitive displacement series (Kim JACS 2001; v0.3.3).

- **Usage Examples:**

```
from synthonicon.models import Fidelity

high_fidelity = Fidelity.HIGH
print(f"High Fidelity: {high_fidelity.value}")

# Accessing from symbol
medium_fidelity = Fidelity.from_symbol("F_-")
print(f"Medium Fidelity from symbol: {medium_fidelity.
      name}")
```


- **Properties:**

- `numeric_value` (float): Returns a standardized numeric value (0.0–1.0) for quantitative calculations.
- `xi_CP_range` (Tuple[float, float]): Provides the typical range for the Inefficiency Index (ξ_{CP}) associated with this fidelity level.

```
print(f"Numeric value for HIGH Fidelity: {high_fidelity.
      numeric_value}")
# Expected: 0.95
print(f"\xi_CP range for MEDIUM Fidelity: {
      medium_fidelity.xi_CP_range} nats")
# Expected: (9.0, 11.5)
```

- `propagate_through(context_factors: Dict[str, float])` Method: This method simulates how fidelity might change when exposed to various environmental or system-level context factors (e.g., `solvent_compatibility`, `temperature_match`, `steric_fit`, `concentration`).

```
context = {"solvent_compatibility": 0.8, "steric_fit":
           0.9}
propagated_fidelity = high_fidelity.propagate_through(
    context)
print(f"Fidelity after propagation: {propagated_fidelity.
      name}")
# Example output: MEDIUM (if context factors reduce
# effective fidelity)
```

0.3.6 Granularity (G)

Granularity defines the scale of control or influence exerted by the synthon within a larger system. It describes whether a synthon dictates organization locally, at an intermediate motif level, or globally across a network.

- **Enum Members:**

- `LOCAL` ("G_beth" or "G_"): Control at a very confined scale, typically involving a single bond or interaction.
- `MESOSCALE` ("G_gimel" or "G_"): Control over a motif or small cluster of interactions (e.g., a specific binding pocket, a small oligomer).
- `GLOBAL` ("G_aleph" or "G_"): Control that extends throughout an entire network or framework (e.g., a crystal lattice, a self-replicating system).

- **Usage Examples:**

```
from synthonicon.models import Granularity

local_gran = Granularity.LOCAL
print(f"Local Granularity: {local_gran.value}")

# Accessing from symbol
```

```
global_gran = Granularity.from_symbol("G_")
print(f"Global Granularity from symbol: {global_gran.name
      }")
```

- **Properties:**

- `scale_factor (int)`: Returns an approximate numerical factor representing the number of components controlled by this granularity level.

```
print(f"Scale factor for MESOSCALE: {Granularity.
      MESOSCALE.scale_factor}")
# Expected: 100
```

- `can_amplify_to(target: Granularity) Method`: Checks if the current granularity level can conceptually "amplify" its control to a specified target granularity level (i.e., local can amplify to mesoscale, mesoscale to global).

```
print(f"Can LOCAL amplify to GLOBAL? {local_gran.
      can_amplify_to(global_gran)}")
# Expected: True
print(f"Can GLOBAL amplify to LOCAL? {global_gran.
      can_amplify_to(local_gran)}")
# Expected: False
```

0.3.7 Interaction Grammar (Γ)

Interaction Grammar governs the logic of partner selection for a synthon. It describes the specificity or promiscuity of its interactions, from highly selective binding to broad, non-specific recognition.

- **Enum Members:**

- **SPECIFIC** ("Gamma_otimes" or " $a_{\otimes}$ "): Selects one highly specific partner, akin to a lock-and-key mechanism.
- **SELECTIVE** ("Gamma_odot" or " $\Gamma_{\cdot}$ "): Interacts with a small, defined set of partners (typically 3-10).
- **BROAD** ("Gamma_bigcirc" or " $\Gamma_{\circ}$ "): Compatible with many potential partners, often leading to less predictable outcomes without additional constraints.
- **Sequential variants (v0.3.5)**: SPECIFIC_SEQ, SELECTIVE_SEQ, BROAD_SEQ -- same specificity tier but operating along a directed temporal axis (D_{∞} systems). Registered as " $a_{\otimes_seq}$ ", " $\Gamma_{\cdot_seq}$ ", " $\Gamma_{\circ_seq}$ ".
- **Dissipative variants (v0.4.0)**: SPECIFIC DISSIPATIVE, SELECTIVE DISSIPATIVE, BROAD DISSIPATIVE -- irreversible coupling governed by $\Gamma_{\cdot}$ (DISSIPATIVE) operator (Lindblad/Zeno channel). Use for open quantum systems where the interaction causes information loss to the environment rather than unitary exchange. `specificity_score` = 0.95 / 0.75 / 0.25 respectively; `partner_count_range` = (1,1) / (1,3) / (1, ∞).
- **Quantum tier (v0.4.0)**: QUANTUM_AND, QUANTUM_OR, QUANTUM_SEQ, QUANTUM DISSIPATIVE -- Toffoli-gate semantics operating on superposition states. QUANTUM_AND requires simultaneous entanglement of two control qubits; QUANTUM_OR

is the logical OR of coherent pathways; QUANTUM_SEQ chains unitary gates preserving superposition; QUANTUM_DISSIPATIVE = measurement/decoherence channel. All quantum members share `partner_count_range = (1, 2)` and `specificity_score = 0.9`. *Identified as missing during quantum particle encoding -- classical Γ values destroy superposition semantics (v0.4.0).*

- Usage Examples:

```
from synthonicon.models import InteractionGrammar

specific_grammar = InteractionGrammar.SPECIFIC
print(f"Specific Grammar: {specific_grammar.value}")

# Accessing from symbol
broad_grammar = InteractionGrammar.from_symbol("\Gamma_")
print(f"Broad Grammar from symbol: {broad_grammar.name}")
```

- Properties:

- `partner_count_range` (Tuple[int, int]): Returns the typical range of compatible partners for this grammar.
- `specificity_score` (float): Provides a numerical score (0.0–1.0) indicating the degree of specificity, where 1.0 is most specific.

```
print(f"Partner count range for SELECTIVE: {
    InteractionGrammar.SELECTIVE.partner_count_range}")
# Expected: (2, 10)
print(f"Specificity score for BROAD: {broad_grammar.
    specificity_score}")
# Expected: 0.1
```

0.3.8 Kinetic Character (K)

Kinetic Character encodes the activation barrier and pathway multiplicity for constraint propagation, independently of thermodynamic fidelity F . A synthon can be F_h (high fidelity) but K_{slow} (kinetically inaccessible), or F_{eth} but K_{fast} .

- Enum Members:

- FAST ("K_fast"): $\Delta G < 60$ kJ/mol; spontaneous on experimental timescales. Accessibility score: 0.95.
- MODERATE ("K_mod"): $\Delta G \approx 60$ –100 kJ/mol; accessible with mild activation. Accessibility score: 0.70.
- SLOW ("K_slow"): $\Delta G > 100$ kJ/mol; requires significant activation. Accessibility score: 0.30.
- TRAP ("K_trap"): Pathway multiplicity high; kinetic products diverge from thermodynamic products. Accessibility score: 0.50.
- MBL ("K_MBL"): NEW in v0.4.0. Many-body localization -- disorder-induced kinetic arrest. The system is frozen not by an energy barrier but by the structure of its many-body eigenbasis; adding energy does

not cause relaxation. Ordinal position 0 (below K_{trap} at 1) in the kinetic hierarchy -- more arrested than any classical trap. Accessibility score: 0.05. Barrier range: (0.0, 0.0) -- not barrier-limited. `meet(K_MBL, anything) -> K_MBL`. **Cannot be inferred from `from_barrier()`**; requires explicit assignment. Physical systems: disordered quantum magnets, cold atoms in quasiperiodic potentials (Aubry-André model), many-body localized phases in 1D interacting systems. *Identified as missing via quantum particle encoding -- K_{trap} is energy-barrier trapping, K_{MBL} is disorder trapping; the distinction matters because MBL stores quantum information indefinitely (v0.4.0).*

- **Example usage:** The carboxylic acid homodimer is F_h , K_{fast} . The gas-phase imine condensation proxy is F_{eth} , K_{slow} ; aqueous imine is F_{eth} , K_{mod} -- same thermodynamic tier, different operational accessibility.

0.3.9 Criticality Phase (Φ)

Criticality Phase encodes the synthon's position relative to the G-D criticality locus.

- **Enum Members:**
 - SUBCRITICAL ("Phi_sub"): G and D are demonstrably independent. Default assignment.
 - CRITICAL ("Phi_c"): At the criticality locus -- $\xi \rightarrow \infty$, scale-free behavior, G/D degenerate (Axiom 5).
 - SUPERCRITICAL ("Phi_super"): Post-assembly state where synthon identity is absorbed into the assembled material.
- **Criticality probe (v2.2+):** Use `syncon criticality-probe <name>` to score Φ_c candidacy. The probe now reports:
 - `z_eff` -- dynamic exponent (diverges logarithmically for Varma QXY class; = 1.33 for 2D percolation)
 - Degeneracy strength score 0-1 with tier: none / logarithmic / power-law / collapse
 - Universality class hint

Eight heuristic factors (v0.4.0):

Factor 8 (v0.4.0) fires when a synthon simultaneously has global granularity, quantum fidelity, and trap-kinetics, but **lacks** D_∞ temporality -- the signature of a quantum critical point without a classical order parameter. Falsifiable prediction: susceptibility $\chi(T \rightarrow 0) \sim T^{-\gamma}$. *Identified as missing when probing the spin singlet and qubit synthons, whose criticality fingerprint is orthogonal to Factors 1-7.*

```
syncon criticality-probe my_synthon --degeneracy-type
syncon criticality-probe --batch --export-candidates
top20.json
```

#	Trigger	Weight	Universality Class
1	G_ (global granularity)	0.15	Varma QXY
2	D_∞ (temporal dimension)	0.15	Varma QXY
3	T_ (self-complementary)	0.15	Varma QXY
4	P_± (symmetric polarity)	0.15	Varma QXY
5	F _h (quantum fidelity)	0.15	Varma QXY
6	db24c8 steric-cliff analog	0.10	Steric-cliff
7	D_∞ + T_ + P_DA + F _h	0.25	Frank-model (classical bifurcation)
8	G_ + F _h + K_trap + ¬D_∞	0.20	Quantum criticality (TFI/heavy-fermion)

0.3.10 Stoichiometry (S)

Stoichiometry encodes the valency ratio of the recognition event -- the molar ratio of partners in the assembled motif. It is a string field (not an Enum) with weight 0.08 in analogy similarity scoring (~6% of total) and hard-constraint enforcement via Pass 4 audit.

- **Value format:** "1:1", "2:1", "3:1", "n:m" (free-form ratio string), or None (unassigned).
- **Consistency rules** (enforced by Pass 4 audit):
 - T_ + S="1:1" → must have P± (self-complementary polarity)
 - T_ + S="n:m" (n≠m) → must have Γ∨(BROAD) or T_network
 - T_ + no S → auto-suggested as "1:1" if P± present; otherwise flagged for manual review
- **Auto-backfill:**

```
# Assign S="1:1" to all T_ + P+/- entries (dry run first)
syncon catalog auto-stoichiometry --dry-run
syncon catalog auto-stoichiometry
# Result: 1,157/1,269 T_ entries auto-fixed; 112 require
manual review
```

- **Similarity grading** in analogy scoring:
- **Python:**

```
synthon = Synthon(name="my_dimer", ..., stoichiometry="
1:1")
```

Case	Score
Exact match	1.00
Both symmetric (n:n)	0.90
Both asymmetric (n≠m)	0.90
Category mismatch, ratio diff < 0.5	0.70
Category mismatch, ratio diff ≥ 0.5	0.20–0.70

```
print(synthon.stoichiometry) # "1:1"
```

0.3.11 Topological Protection Index (Ω) -- 11th Primitive (v0.4.0)

Ω encodes the Altland-Zirnbauer / K-theory topological classification of a synthon. It is an optional field that defaults to None for all classical (non-topological) synthons. When set, it appears as the 11th element of the tuple notation: $\langle \dots; S; \Omega \rangle$.

- **Enum:** TopoIndex (importable from synthomnicon)
- **Enum Members** (ordered by protection strength, ordinal 0→4):

Symbol	Value	Ordinal	Description	Physical Systems
Ω_0	TopoIndex.TRIVI	0	No topological protection; fully classical	Ordinary insulators, non-topological magnets
Ω_Z	TopoIndex.Z2	1	protection; time-reversal-symmetric TIs	3D TIs (BiSe), QSHI, Z SPT phases
Ω_Z	TopoIndex.Z_CLA	2	classification; integer invariant	Integer QHE ($\sigma_{xy} = ne^2/h$), Chern insulators
Ω_{Ch}	TopoIndex.CHERN	3	Chern-number protection; broken time-reversal	Haldane model, magnetic TIs, anomalous QHE
Ω_{NA}	TopoIndex.NON_A	4	Non-Abelian anyonic statistics; braided exchange	Kitaev honeycomb, Moore-Read FQH ($\nu=5/2$), Majorana platforms

- **Lattice semantics:**
 - `meet( $\Omega_a$ ,  $\Omega_b$ ) ->  $\Omega_{min}$` -- conservative guarantee: use the weaker protection class when two systems couple

- `join( $\Omega_a$ ,  $\Omega_b$ ) ->  $\Omega_{\max}$` -- capability ceiling: higher class can host everything the lower class can
- `tensor( $\Omega_a$ ,  $\Omega_b$ ) ->  $\Omega_{\max}$` -- composite system inherits the stronger topological protection
- **Distance weight:** 0.7 in `tuple_distance()` -- Ω contributes ~5.7% of total distance (weight 0.7 out of default total ≈ 12.3).
- **Usage:**

```
from synthomnicon import TopoIndex, Synthon

kitaev_chain = Synthon(
    name="kitaev_chain",
    ...,
    topo_index=TopoIndex.NON_ABELIAN,    # \Omega_{NA} ---
    Majorana edge modes
)

print(kitaev_chain.topo_index.protection_strength)    # 4
print(kitaev_chain.topo_index.physical_systems)
# -> 'Kitaev honeycomb, Moore-Read FQH ( $\nu=5/2$ ), Majorana
      platforms'
print(kitaev_chain.to_notation())
# -> \langle...; \Omega_{NA}\rangle
```

- **syncon analyze display** (when Ω is set):

```
\Omega (TopoIndex)    \Omega_{NA}    Non-Abelian anyonic .
protection_strength=4
```

- *Identified as missing when encoding the five quantum particle synthons (photon/proton/electron/spin/qubit). The framework had no way to distinguish a trivially gapped insulator from a topological insulator or non-Abelian fractional QH state -- the distinction is operationally essential for quantum information protection arguments (v0.4.0).*

0.4 Command Line Interface (CLI)

SynthOmnicon provides a powerful command-line interface for rapid analysis and exploration of synthons. The CLI is accessible via `main.py` or the `synthomnicon` command (if installed as a package).

0.4.1 Core Commands

analyze Analyze a synthon by its name or a raw notation string.

Usage:

```
python3 main.py analyze [IDENTIFIER]
```

Example:

```
python3 main.py analyze carboxylic_acid_dimer
```

catalog list List all synthons currently registered in the global catalog.

Usage:

```
python3 main.py catalog list [--domain DOMAIN]
```

Example:

```
python3 main.py catalog list --domain temporal
```

thermo Calculate thermodynamic metrics (η_{CP} and ξ_{CP}) for a registered synthon.

Usage:

```
python3 main.py thermo [IDENTIFIER] --delta-g [KJ_MOL]
```

Example:

```
python3 main.py thermo carboxylic_acid_dimer --delta-g -12.0
```

check Check the compatibility between two synthons.

Usage:

```
python3 main.py check [SYNTHON_A] [SYNTHON_B]
```

Example:

```
python3 main.py check enolate_synthon carbonyl_synthon
```

0.4.2 Catalog Audit Command

The syncon audit command runs a 4-pass scan of the catalog for grounding issues, topology errors, attractor-tuple contamination, and stoichiometry inconsistencies.

Pass summary:

Usage:

```
syncon audit [OPTIONS]
```

Options:

Examples:

Pass	Axiom	Checks
1	Axiom 6	D _∞ entries: structured grounding["reset"] block checked first (discrete or continuous); falls back to keyword scan
2	Axiom 7	T _∞ entries for named closing bond; rejects linear/rod/chain
3	—	Attractor-tuple contamination ($\geq 7/7$ match, no stored reasoning)
4 (<i>v2.2</i>)	—	S consistency: T _∞ [1:1] ↔ P±; T _∞ [n:m] ↔ Γ ∨ BROAD/T _∞ network

Option	Description
-axiom 6	Audit all D _∞ entries for Axiom 6 (discrete reset or continuous dissipative flux)
-axiom 7	Audit all T _∞ entries for Axiom 7 (named closing bond)
-axiom 4 (<i>v2.2</i>)	Audit T _∞ entries for stoichiometry consistency (Pass 4)
-primitive D/T/R/...	Filter by primitive type
-value TEXT	Filter by primitive value (e.g. temporal , cyclic)
-status STATUS	Filter by grounding status (unverified, partial, override, full, flagged_for_review)
-auto-flag	Set grounding_status="flagged_for_review" on problem entries
-dry-run	Preview what would be flagged without making changes

```

# Audit all D_inf entries for closed-cycle grounding
syncon audit --axiom 6

# Audit all T_ entries and auto-flag those without closing bond
syncon audit --axiom 7 --auto-flag

# Preview unverified entries without making changes
syncon audit --status unverified --dry-run

# Find all temporal entries with partial grounding
syncon audit --primitive D --value temporal --status partial

# Full audit: all unverified entries across catalog
syncon audit --status unverified --auto-flag

```

Understanding the output:

The command prints a Rich table with columns:

- **Synthon** -- catalog entry name
- **Domain** -- molecular / supramolecular / temporal / speculative
- **Grounding Status** -- full / partial / override / unverified / flagged_for_review
- **Failed Primitives** -- primitives that failed grounding checks
- **Flag Reason** -- why the entry was flagged (grounding status or axiom check)

Axiom 6 check logic (updated v0.3.3 -- discrete/continuous):

The audit calls `AxiomValidator.validate_axiom6_temporal_grounding(synthon)` which checks in priority order:

1. • **Structured block** -- `synthon.grounding["reset"]` (persisted in catalog JSON, survives reload): `type = "discrete":` requires `cycle_steps` list ≥ 2 , or `axiom6_grounding` metadata block
 - `type = "continuous":` requires `driving_gradient.description + driving_gradie`
2. **Keyword scan fallback** -- only when no structured block exists. Scans `description/reasoning` for `AXIOM_6_RESET_INDICATORS + AXIOM_6_PROCESS_INDICATORS`; both must be present.

Adding structured Axiom 6 grounding to a catalog entry:

```

from synthonicon import global_catalog

s = global_catalog.get("my_dissipative_synthon")

# Continuous (dissipative, far-from-equilibrium):
s.grounding["reset"] = {
    "type": "continuous",
    "driving_gradient": {

```

```

        "description": "Continuous reagent supply (fuel +
        substrate) drives steady-state turnover",
        "coupling": "Chemical potential gradient: fuel ->
        product, with autocatalytic product feedback",
        "entropy_export": "Heat dissipation to solvent;
        byproduct efflux",
    },
    "termination_condition": "Reagent depletion ends the cycle
    ; no sharp reset event.",
}

# Discrete (closed cycle with explicit reset step):
s.grounding["reset"] = {
    "type": "discrete",
    "cycle_steps": [
        "Step 1: catalyst activates substrate",
        "Step 2: product formed",
        "Step 3: hydrolysis/relaxation returns catalyst to
        initial state (reset)",
    ],
}

global_catalog.save_catalog()

```

Known grounded entries (v0.3.3):

Entry	Reset type	Mechanism
soai_pyrimidyl_autocataly	continuous	iPrZn + aldehyde flux; Frank-model bifurcation
proline_aldol_cycle	discrete	Iminium hydrolysis → free proline reset

Axiom 7 check logic: Scans the description for AXIOM_7_CLOSING_INDICATORS (e.g. "hydrogen bond", "ring", "macrocycle") and rejects if AXIOM_7_INVALID_TOPO_KEY are found (e.g. "linear", "rod", "allene").

0.4.3 I(bits) Calibration Command (v2.2)

```

# Run full calibration pipeline (vacuum)
syncon info-bits --calibrate

# With solvent correction
syncon info-bits --calibrate --solvent chloroform
syncon info-bits --calibrate --solvent water

# Single synthon
syncon info-bits carboxylic_acid_dimer
syncon info-bits triple_hbond_array --solvent THF

```

Output fields: I_recognition, I_orientation (overhead), I_net = I_rec - 0.3xI_orientation, I_total+solvent, DeltaS_conf, heuristic comparison, verdict.

Calibrated ranges (v2.2):

System type	I_recognition
2-HB dimers (acid dimer)	9–10 bits
Cooperative arrays (triple H-bond DAD·ADA)	14–18 bits
Quadruple arrays (AADD·DDAA)	19–24 bits
Temporal cycles (proline aldol)	6–9 bits/turn

0.4.4 Criticality Probe Command (v2.2 / updated v0.3.3)

```
# Single entry
syncon criticality-probe my_synthon
syncon criticality-probe my_synthon --degeneracy-type

# With measured correlation lengths (Varma QXY test)
syncon criticality-probe my_synthon --xi-r 6.2 --xi-tau 1.8e14

# Batch mode: scan all Phi_c / T_+D_inf / steric-cliff
  candidacy entries
syncon criticality-probe --batch
syncon criticality-probe --batch --degeneracy-type
syncon criticality-probe --batch --export-candidates
  candidates.json
```

Degeneracy tiers:

Score	Tier	Meaning
0.00–0.30	none	G and D independent
0.30–0.60	logarithmic	Varma QXY — $\xi_r \approx \ln i_\tau$
0.60–0.85	power-law	Conventional QCP — finite z
0.85–1.00	collapse	Direct G/D identity

Scoring factors (score_phi_c_candidacy(), varma_probe.py):

Validated case studies (v0.3.3):

ω_c normalization note: $i_\tau = \tau_{\text{corr}} \times \omega_c$ where ω_c is the appropriate natural frequency for the system's relevant timescale. For solution-phase catalytic systems, use $\omega_c \approx 10\tau^2 \text{ s}^{-1}$ (solvent relaxation frequency) rather than $k_B T/h \approx 6.25 \times 10^{12} \text{ s}^{-1}$ to avoid mixing molecular-vibration and macroscopic-catalytic timescales.

Factor	Weight	Fires when	Universality class	
1–5 (Varma QXY structural heuristics)	0.35 / 0.25 / 0.20 / ...	D_{∞} , R_{∞} , G_{∞} primitives present	Varma QXY (quantum)	
6 (Steric-cliff proxy)	0.65	<code>phi_c_candidacy.prc</code> ≥ 0.50 in grounding	Steric-cliff (mechanical)	gth
7 (Frank-model classical bifurcation)	0.25	$D_{\infty} + T_{\infty} + P_{\text{directional}} + F_h$ all present	Frank 1953 (classical pitchfork)	

System	ξ_r	i_r	Ratio	Score	Result
Soai autocatalytic cycle	15	7.2×10^2	0.94	0.920	Approaching Φ_c (Frank model)
DB24C8 pseudorotaxane	—	—	—	0.461	Approaching Φ_c (steric cliff)
Proline-aldol cycle	6.2	1.8×10^2	0.189	0.380	Φ_{sub} (subcritical, confirmed)

0.4.5 Stoichiometry Commands (v2.2)

```
# Auto-assign S="1:1" to T_ + P+/- entries
syncon catalog auto-stoichiometry --dry-run      # preview
syncon catalog auto-stoichiometry                 # apply (up to
500 entries)
syncon catalog auto-stoichiometry --limit 2000    # larger batch

# Analogy search with stoichiometry flags
syncon analogies my_synthon --stoichiometry-aware # raises S
weight to 0.12
syncon analogies my_synthon --critical-only        # pre-
filter by Phi_c score > 0.5
syncon analogies my_synthon --stoichiometry-aware --critical-
only
```

0.4.6 Experimental Validation Case Studies (v0.3.3)

CB[7] Competitive Displacement — F-floor Ratchet (6/6 Validated) Tests the HotSwap F-floor hard constraint: a swap is BLOCKED if $F_{\text{new}} < F_{\text{old}}$. The CB[7] host-guest series provides a literature-grounded three-tier ordering.

```
# Register the three CB[7] complexes (already in catalog post-
v0.3.3)
# CB7_ferrocene_ammonium_complex ($F_{\hbar}$, Ka =  $3 \times 10^2$ ,
DeltaG = -71.2 kJ/mol)
# CB7_adamantane_ammonium_complex (F_, Ka =  $4.3 \times 10$ , DeltaG =
```

```

-49.1 kJ/mol)
# CB7_DABCO_complex ($F_{\ell}$, Ka = 2x10,
DeltaG = -30.1 kJ/mol)

# Run all 6 directional swaps --- expected: 3 APPROVED, 3
BLOCKED
syncon hotswap CB7_ferrocene_ammonium_complex
CB7_adamantane_ammonium_complex --delta-g -49.1
# -> APPROVED ($F_{\hbar}$ -> F_, downgrade: F_new=F_ < F_old=
$F_{\hbar}$) <- NOTE: Fc->Ad is BLOCKED
syncon hotswap CB7_adamantane_ammonium_complex
CB7_ferrocene_ammonium_complex --delta-g -71.2
# -> APPROVED (Ad->Fc: F_new=$F_{\hbar}$ > F_old=F_)

syncon hotswap CB7_DABCO_complex
CB7_adamantane_ammonium_complex --delta-g -49.1
# -> APPROVED (DABCO->Ad: F_new=F_ > F_old=$F_{\ell}$)
syncon hotswap CB7_adamantane_ammonium_complex
CB7_DABCO_complex --delta-g -30.1
# -> BLOCKED (Ad->DABCO: F_new=$F_{\ell}$ < F_old=F_)

syncon hotswap CB7_ferrocene_ammonium_complex
CB7_DABCO_complex --delta-g -30.1
# -> BLOCKED (Fc->DABCO: F_new=$F_{\ell}$ < F_old=$F_{\hbar}$)
syncon hotswap CB7_DABCO_complex
CB7_ferrocene_ammonium_complex --delta-g -71.2
# -> APPROVED (DABCO->Fc: F_new=$F_{\hbar}$ > F_old=$F_{\ell}$
)

```

Result table (all 6 match experiment):

Swap	Framework	Experiment (Kim 2001)
Fc displaces Ad	APPROVED	Yes (quantitative)
Ad displaces Fc	BLOCKED	No (no exchange)
Ad displaces DABCO	BLOCKED	No (DABCO weaker → Ad is stronger, Ad stays)
DABCO displaces Ad	APPROVED	Yes (Ad quantitatively displaces DABCO, confirmed by reverse)
Fc displaces DABCO	BLOCKED	No
DABCO displaces Fc	APPROVED	Yes

References: Kim et al. J. Am. Chem. Soc. 123, 12091 (2001); Assaf & Nau, Chem. Soc. Rev. 44, 394 (2015); Sindelar et al. J. Org. Chem. 72, 3221 (2007).

Soai Reaction Varma Probe — Approaching Φ_c (Score 0.920)

```
syncon criticality-probe soai_pyrimidyl_autocatalytic_cycle --
xi-r 15 --xi-tau 7.2e15
```

Expected output summary:

- Ratio $\xi_r / \ln(i_\tau) = 15 / \ln(7.2 \times 10^2) \approx 0.94$ (near-critical; threshold = $1.0 \pm 20\%$)
- Factor 7 fires: $D_\infty + T_- + P_{\text{directional}} + F_h \rightarrow$ Frank-model pitchfork bifurcation
- Score: 0.920 -- approaching Φ_c
- Active species: $[\text{Zn} \cdot (\text{pyrimidylalkoxide}) \cdot \text{iPrZn}]$ dimer (Gridnev 2010)

Proline-Aldol Varma Probe — Φ_{sub} Confirmed (Score 0.380)

```
syncon criticality-probe proline_alcohol_cycle --xi-r 6.2 --xi-
tau 1.8e14
```

Expected output summary:

- Ratio $\xi_r / \ln(i_\tau) = 6.2 / \ln(1.8 \times 10^2) \approx 0.189$ (well below 1.0 threshold)
- Factor 7 does not fire: $P_{\pm}^{\psi}$ (pseudosymmetric, not directional P_{DA})
- Score: 0.380 -- Φ_{sub} (subcritical)
- Structural prediction: criticality would require $\xi_r \geq 32$ lattice units (~48 Å correlation), incompatible with observed enamine geometry

0.5 Building and Managing Synthons

The core data structure in SynthOmnicon is the Synthon class. It encapsulates all ten primitives and provides methods for notation generation and serialization.

0.5.1 Creating Synthons

You can create a Synthon object by providing its name and primitives as enum members plus the optional stoichiometry string.

```
from synthomnicon import (
    Synthon, Dimensionality, Topology, RecognitionMode,
    Polarity, Fidelity, KineticCharacter, Granularity,
    InteractionGrammar, CriticalityPhase
)

carboxylic_dimer = Synthon(
    name="carboxylic_acid_dimer",
    dimensionality=Dimensionality.MOLECULAR,
    topology=Topology.CYCLIC_BOWTIE,
    recognition_mode=RecognitionMode.NON_COVALENT,
    polarity=Polarity.SELF_COMPLEMENTARY,
    fidelity=Fidelity.HIGH,
    kinetic_character=KineticCharacter.FAST,
    granularity=Granularity.LOCAL,
    interaction_grammar=InteractionGrammar.SPECIFIC,
    criticality_phase=CriticalityPhase.SUBCRITICAL,
```

```

    stoichiometry="1:1",
    description="Classic hydrogen-bonded dimer (R^2(8) motif)"
)

```

0.5.2 Unified Notation

The `to_notation()` method generates the full formal string: `; T; R; P; F; K; G; Γ; Phi; S`.

```

print(carboxylic_dimer.to_notation())
# Output: \langle D_wedge; T_bowtie; R_superset; P_pm; F_hbar;
          K_fast; G_beth; Gamma_otimes; Phi_sub; 1:1 \rangle

```

You can also parse a notation string back into a `SynthonNotation` object:

```

from synthonnicon import parse_notation
notation = parse_notation(
    "\langle D_wedge; T_bowtie; R_superset; P_pm; F_hbar;
     K_fast; G_beth; Gamma_otimes; Phi_sub; 1:1 \rangle"
)
print(notation.dimensionality) # Dimensionality.MOLECULAR
print(notation.stoichiometry)  # "1:1"

```

0.5.3 Serialization

Synthons support serialization to dictionaries and JSON for easy storage and transmission.

```

# To Dictionary
data = carboxylic_dimer.to_dict()

# To JSON
json_str = carboxylic_dimer.to_json()

# From JSON
new_synthon = Synthon.from_json(json_str)

```

0.6 SynthonCatalog: Storage and Discovery

The `SynthonCatalog` acts as a centralized repository for synthons, enabling advanced search and cross-domain reasoning.

0.6.1 Registration

Registering synthons allows you to retrieve them by name later or include them in global searches.

```

from synthonnicon.registry import global_catalog

# The catalog is automatically populated with defaults on CLI
start,

```



```
# but you can register your own:
global_catalog.register(carboxylic_dimer)
```

0.6.2 Searching

The catalog supports indexed searches by any of the ten primitives.

```
# Search for all high-fidelity synthons
high_f = global_catalog.search(fidelity=Fidelity.HIGH)

# Search for all temporal synthons
temporal = global_catalog.search_by_domain("temporal")
```

0.6.3 Cross-Domain Analogy Search

One of the most powerful features of SynthOmnicon is the ability to find "analogs" across different domains (e.g., finding a temporal process that behaves like a molecular H-bond dimer).

```
# Find temporal synthons that are topologically and
functionally
# analogous to a molecular carboxylic acid dimer
analogs = global_catalog.find_cross_domain_analogs(
    carboxylic_dimer,
    target_domain="temporal"
)
```

0.7 Constraint Propagation Engine

The ConstraintEngine and FidelityPropagator are used to analyze how synthons interact and how their individual constraints combine to influence the overall system.

0.7.1 Pairwise Compatibility

Check if two synthons can interact based on their primitives (recognition mode, polarity, dimensionality, etc.).

```
from synthomnicon.constraints import ConstraintEngine

engine = ConstraintEngine()
report = engine.check_pair_compatibility(synthon_a, synthon_b)

if report.is_compatible:
    print(f"Compatible! Shared domains: {report.details['shared_domains']}")
else:
    print(f"Incompatible: {report.result.name}")
```

0.7.2 Fidelity Propagation and Cooperativity

Individual synthon fidelities often amplify when combined in specific topologies (e.g., cyclic motifs show superlinear cooperativity).

```
from synthomnicon.constraints import FidelityPropagator

propagator = FidelityPropagator()

# Propagate fidelity across a list of interacting synthons
total_fidelity = propagator.propagate([synthon_a, synthon_b,
    synthon_c])

# Compute cooperativity factors
coop = propagator.compute_cooperativity_factor([synthon_a,
    synthon_b])
print(f"Total cooperativity: {coop['total_cooperativity']:.2f}
    ")
```

0.8 Thermodynamics: Quantifying Efficiency

SynthOmnicon introduces formal metrics to quantify how effectively physical energy (ΔG) is converted into structural information (I) with a given reliability (F).

0.8.1 η_{CP} and ξ_{CP}

- η_{CP} (Efficiency): The ratio of reliable information gain to the energy cost (relative to the Landauer limit).
- ξ_{CP} (Inefficiency Index): The negative log of efficiency, measured in nats.

```
from synthomnicon.thermodynamics import compute_eta_CP

# Calculate metrics for a carboxylic acid dimer (DeltaG(298K,
    gas) = -12 kJ/mol)
result = compute_eta_CP(carboxylic_dimer, delta_g=-12.0)

print(f"Efficiency (\eta_CP): {result.eta_CP:.2e}")
print(f"Inefficiency (\xi_CP): {result.xi_CP:.2f} nats")
print(f"Assessment: {result.assessment}")
```

0.8.2 Landauer Benchmarking

Compare your system's efficiency directly to the theoretical minimum energy required to process the equivalent amount of information.

```
from synthomnicon.thermodynamics import
    benchmark_against_landauer

benchmark = benchmark_against_landauer(carboxylic_dimer,
    delta_g=-12.0)
```

```
print(f"Overhead: {benchmark['overhead_ratio']:.1e}x Landauer
      limit")
```

0.9 Domain-Specific Agents

SynthOmnicon includes specialized agents that apply the core framework to specific chemical domains.

0.9.1 Molecular Agent

Used for retrosynthetic analysis and molecular bond disconnection.

```
from synthomnicon.domains.molecular import
    MolecularSynthonAgent

mol_agent = MolecularSynthonAgent()
analysis = mol_agent.analyze_reaction_center("CC(=O)O")
print(f"Disconnection Feasibility: {analysis.
      disconnection_feasibility}")
```

0.9.2 Supramolecular Agent

Focuses on spatial organization and cooperative effects in non-covalent networks.

```
from synthomnicon.domains.supramolecular import
    SupramolecularSynthonAgent

supra_agent = SupramolecularSynthonAgent()
coop = supra_agent.compute_cooperativity_induction(num_bonds
    =3)
```

0.9.3 Temporal Agent

Analyzes time-dependent processes, catalytic cycles, and temporal reliability.

```
from synthomnicon.domains.temporal import TemporalSynthonAgent

temp_agent = TemporalSynthonAgent()
fidelity = temp_agent.compute_fidelity_per_cycle(k_cat=1.0,
    k_side=0.001)
```

0.10 AI-Powered Synthon Generation

SynthOmnicon includes an LLM-powered agent that can automatically generate synthons from natural language descriptions or SMILES strings.

0.10.1 SynthonGeneratorAgent

The SynthonGeneratorAgent uses advanced language models to analyze chemical descriptions and map them to all ten primitives.

```

import asyncio
from agents.synthon_generator_agent import
    SynthonGeneratorAgent
from synthonmicon.provider_config import build_agent_config

async def main():
    # Use config-driven defaults (model=None uses provider
    # default)
    config = build_agent_config(provider="anthropic", model=
    None)
    agent = SynthonGeneratorAgent(config)

    # Generate from natural language description
    result = await agent.generate_from_description(
        "carboxylic acid dimer with cyclic hydrogen bonding",
        delta_g=-12.0, # Optional: for thermodynamic analysis
        # (DeltaG(298K, gas) for acid dimer)
        auto_register=True # Automatically add to catalog
    )

    print(f"Generated: {result.synthon.name}")
    print(f"Notation: {result.synthon.to_notation()}")
    print(f"Confidence: {result.confidence:.1%}")
    print(f"Reasoning: {result.reasoning}")

    # Access thermodynamic metrics if delta_g was provided
    if result.thermodynamic_metrics:
        print(f"\eta_CP: {result.thermodynamic_metrics['eta_CP
        ']:.2e}")
        print(f"\xi_CP: {result.thermodynamic_metrics['xi_CP
        ']:.4f} nats")

asyncio.run(main())

```

0.10.2 Generation from SMILES

```

from agents.synthon_generator_agent import
    SynthonGeneratorAgent
from synthonmicon.provider_config import build_agent_config

config = build_agent_config(provider="anthropic", model=None)
agent = SynthonGeneratorAgent(config)

result = await agent.generate_from_smiles(
    "CC(=O)O", # Acetic acid
    name="acetic_acid",
    functional_groups=["carboxylic_acid"],
    auto_register=True
)

```

```
print(f"Generated synthon: {result.synthon.to_notation()}")
```

0.10.3 Convenience Function

For quick synthon generation, use the `generate_synthon` function:

```
from agents.synthon_generator_agent import generate_synthon

result = await generate_synthon(
    "DNA adenine-thymine base pair",
    provider="qwen",
    model="qwen3-max",
    delta_g=-80.0
)

print(f"Notation: {result.synthon.to_notation()}")
print(f"\xi_CP: {result.thermodynamic_metrics['xi_CP']:.4f} nats")
```

0.10.4 CLI Commands for AI Generation

The CLI is accessible via both `synthomnicon` and the short alias `syncon`:

```
# Both commands are equivalent:
synthomnicon generate "carboxylic acid dimer" --delta-g -12.0
syncon generate "carboxylic acid dimer" --delta-g -12.0

# Generate from SMILES string
syncon generate-smiles "CC(=O)O" --name acetic_acid

# Specify provider and model
syncon generate "proline catalyzed aldol cycle" \
    --provider qwen \
    --model qwen3-max

# Save result to file
syncon generate "triple hydrogen bond array" \
    --output result.json

# Generate without auto-registration
syncon generate "test system" --no-register
```

Grounding-Controlled Registration (Fix 1 — NEW in v2.1.3)

```
# Block registration if grounding validation fails
syncon generate "carboxylic acid dimer" --strict-grounding

# Force registration despite failures --- logs to audit trail
syncon generate "novel quantum system" \
    --override-grounding \
```

```

--override-reason "speculative entry, grounding pending
publication"

# Register in the 'speculative' domain (Fix 5 quarantine)
syncon generate "quantum tunneling synthon" --speculative

# Combine: speculative + override
syncon generate "..." --speculative --override-grounding \
    --override-reason "theoretical proposal"

```

Grounding flag semantics:

Flag	Effect
-strict-grounding	Raises GroundingValidationError if any primitive fails grounding
-override-grounding	Allows registration despite failure — requires -override-reason
-override-reason TEXT	Justification string logged to audit trail
-speculative	Sets domain="speculative" in CatalogEntry

0.10.5 Agent Framework CLI

The AjintK agent framework is accessible via the agents subcommand:

```

# List all available agents
syncon agents list

# Run SynthonGeneratorAgent from CLI
syncon agents run -d "carboxylic acid dimer" -g -12.0
syncon agents run --provider qwen --model qwen3-max -d "DNA
    base pair"

# Generate from SMILES via agent
syncon agents from-smiles "CC(=O)O" --name acetic_acid
syncon agents from-smiles "CC(=O)O" -o result.json

```

0.10.6 Additional CLI Commands

Compare Synthons

```

# Compare multiple synthons side-by-side
synthomnicon compare carboxylic_acid_dimer
    adenine_thymine_pair

# Include thermodynamic comparison
synthomnicon compare synthon1 synthon2 --delta-g -52.0 -45.0
    --include-thermo

```

Catalog Tree View

```
# View catalog as hierarchical tree
synthomnicon catalog tree

# Filter by domain
synthomnicon catalog tree --domain molecular
```

Export Catalog

```
# Export to JSON
synthomnicon export --format json --output synthons.json

# Export to CSV
synthomnicon export --format csv --output synthons.csv

# Export specific domain
synthomnicon export --domain temporal --format yaml
```

0.11 Autonomous Synthon Discovery

The `AutonomousSynthonDiscoveryAgent` enables continuous, self-directed synthon discovery. The agent autonomously proposes novel chemical systems, validates them against existing knowledge, and registers valid synthons to the persistent catalog.

0.11.1 Running Autonomous Discovery

```
# Basic: 10 cycles, 30 minutes
syncon agents discover

# Extended campaign: 100 cycles, 2 hours
syncon agents discover --cycles 100 --duration 120

# Focused exploration: halogen bonding
syncon agents discover --focus "halogen bonding" --cycles 50

# Different provider with lower confidence threshold
syncon agents discover --provider qwen --confidence 0.6 --
    cycles 30
```

0.11.2 Python API

```
import asyncio
from agents.autonomous_synthon_discovery_agent import (
    AutonomousSynthonDiscoveryAgent,
    AutonomousRunConfig,
    run_autonomous_discovery,
)
from synthomnicon.provider_config import build_agent_config

# Method 1: Convenience function
```

```

results = await run_autonomous_discovery(
    max_cycles=50,
    max_minutes=60,
    provider="anthropic",
    focus="catalysis",
)

# Method 2: Full control
config = build_agent_config(provider="anthropic", model=None)
agent = AutonomousSynthonDiscoveryAgent(config)

run_config = AutonomousRunConfig(
    max_cycles=100,
    max_duration_minutes=120,
    min_confidence_threshold=0.7,
    target_domains=["molecular", "supramolecular"],
    focus_areas=["halogen bonding", "chalcogen bonding"],
    save_interval=10,
)

results = await agent.run_autonomous(run_config)

# Access results
print(f"Cycles completed: {agent.stats['cycles_completed']}")
print(f"Synthons registered: {agent.stats['synthons_registered']}")
print(f"Success rate: {agent.stats['synthons_registered'] /
    agent.stats['cycles_completed'] * 100:.1f}%")

```

0.11.3 Configuration Options

Parameter	Type	Default	Description
max_cycles	int	100	Maximum discovery cycles
max_duration_minute	float	60.0	Maximum runtime
min_confidence_thre	float	0.7	Minimum confidence for registration
target_domains	List[str]	All	Domains to explore
focus_areas	Optional[List[str]]	None	Specific chemistry focus
save_interval	int	10	Save progress every N cycles
output_dir	Optional[Path]	None	Output directory
diversity_mode	bool	True	Actively avoid repeats

0.11.4 Output Files

The agent saves to `./discovery_output/` (or custom `-output`):

- `discovery_history_*.json` -- Cycle-by-cycle results
- `discovery_stats_*.json` -- Statistics and configuration
- `catalog_*.json` -- Catalog export at checkpoint

0.11.5 Validation Results

Result	Icon	Description
<code>valid_novel</code>		Successfully registered
<code>duplicate_exists</code>	⊗	Already in catalog
<code>invalid_chemistry</code>		Chemically invalid or axiom violation
<code>literature_conflict</code>		Conflicts with known systems
<code>low_confidence</code>	?	Below threshold

0.11.6 Axiom Validation (March 14, 2026)

All synthons are now validated against composition axioms before registration. This prevents physically impossible primitive combinations from being registered, ensuring catalog integrity.

Hard constraints (automatic rejection):

1. **Axiom 1 (Cyclic Closure):** Cyclic self-complementary synthons ($T/P_{\pm}$) cannot have low fidelity (F_{ell})
 - *Rationale:* Cyclic closure amplifies fidelity through cooperativity
 - *Example rejection:* $T_{bowtie} + P_{pm_sym} + F_{ell} \rightarrow \text{rejected}$
2. **Axiom 4 (Sequential Grammar):** Sequential grammar ($\Gamma_{\rightarrow}$) requires temporal (D_{∞}) or catalytic ($R_{\rightarrow}$) dimension
 - *Rationale:* Ordered recognition requires state change mechanism
 - *Example rejection:* $D_{triangle} + R_{superset} + \Gamma_{seq} \rightarrow \text{rejected}$
 - *Example acceptance:* $D_{infinity} + R_{superset} + \Gamma_{seq} \rightarrow \text{accepted}$ (has temporal dimension)

Soft constraints (flagged for review):

- Axiom 2 (Local Grammar Barrier)
- Axiom 3 (Cooperative Induction)
- Axiom 5 (Criticality)

Synthons with soft axiom violations are registered but flagged in metadata:

```
synthon.metadata["axiom_validated"] = True
synthon.metadata["axiom_violations"] = 1 # Count of soft
violation
synthon.metadata["axiom_warnings"] = ["Axiom 2 violation
detected - flagged for review"]
```

Viewing axiom validation results:

```
for result in results:
    if result.validation_result == ValidationResult.
INVALID_CHEMISTRY:
        if "AXIOM" in result.reasoning:
            print(f"Axiom violation: {result.reasoning}")

        if result.synthon and result.synthon.metadata.get("
axiom_warnings"):
            print(f"Warning: {result.synthon.name} - {result.
synthon.metadata['axiom_warnings']}")
```

References:

- See AXIOM_VALIDATION_FIX.md for full technical details
- See QUANTSYNTHONICON.md Section IV for axiom definitions
- See test_axiom_validation.py for validation test suite

0.11.7 Example Session

```
$ syncon agents discover --focus "nitroso radicals" --cycles
20
```

```
=====
AUTONOMOUS SYNTHON DISCOVERY AGENT
=====
```

```
Configuration:
  Max cycles: 20
  Max duration: 30.0 minutes
  Min confidence: 0.7
  Focus areas: ['nitroso radicals']
=====
```

```
=====
CYCLE 1/20
=====
```

```
[ ] valid_novel
  Name: nitroso_radical_halogen_bonding_synthon_pair
  Notation: \langleD_wedge; T_linear; R_superset; P_pm;
F_eth; K_fast; G_beth; Gamma_otimes; Phi_sub; 1:1\rangle
  Confidence: 80.0%
=====
```

```

CYCLE 2/20
=====

[] valid_novel
  Name: nitroso_radical_anion_pi_synthon_pair
  Notation: \langleD_wedge; T_linear; R_superset;
  P_directional; F_hbar; K_fast; G_beth; Gamma_otimes;
  Phi_sub; 1:1\rangle
  Confidence: 90.0%

...

```

```

=====

DISCOVERY RUN COMPLETE
=====

```

```

Duration: 3.2 minutes
Cycles completed: 20
Synthons registered: 15
Success rate: 75.0%

```

```

Catalog now contains 1,298 synthons
=====

```

0.11.8 Catalog Persistence

All synthons are automatically persisted to `~/.synthomnicon/catalog.json`:

- **Auto-save:** After each registration
- **Auto-load:** On startup
- **Permanent:** Survives across sessions

```

from synthomnicon import global_catalog

# Catalog persists across runs
print(f"Catalog contains {len(global_catalog)} synthons")

# Access auto-discovered synthons
auto_synthons = [
    s for s in global_catalog._synthons.values()
    if s.metadata.get('auto_discovered')
]
print(f"Auto-discovered: {len(auto_synthons)}")

```

0.12 Integration with AjintK Multi-Agent Framework

SynthOmnicon is built on top of the AjintK multi-agent framework, allowing for complex, distributed chemical reasoning.

0.12.1 Orchestrator Usage

The AgentOrchestrator manages multiple agents and their communication.

```
from framework.orchestrator import AgentOrchestrator
from synthonicon.domains.molecular import
    MolecularSynthonAgent

orchestrator = AgentOrchestrator()
orchestrator.register_agent("molecular_expert",
    MolecularSynthonAgent())

# Run a task through the orchestrator
result = asyncio.run(orchestrator.run_agent(
    "molecular_expert",
    task="Analyze bond disconnections for caffeine"
))
```

0.13 Advanced Topics and Troubleshooting

0.13.1 Best Practices

- **Registry over Raw Objects:** Always prefer registering synthons in the `global_catalog` to enable cross-domain reasoning features.
- **Fidelity Calibration:** Fidelity values should be calibrated against known experimental benchmarks (see `QUANTSYNTHONICON.md`).
- **Domain Overlap:** When modeling complex materials like MOFs, use hybrid dimensionality synthons to capture both molecular and supramolecular constraints
- **AI Generation Confidence:** Review the confidence scores and reasoning from AI-generated synthons before using them in critical analyses.

0.13.2 Troubleshooting

- **NameError: name 'cls' is not defined:** Ensure you are using the latest version of the framework where enum property references have been fixed.
- **ValueError: mutable default ... not allowed:** Occurs if dataclasses use shared dicts. Use `default_factory` for all dictionary fields.
- **Integration Test Failures:** Run `python3 test_integration.py` to diagnose issues. Check your Python version (3.12+ recommended).
- **AI Generation Errors:** Ensure API keys are set for your chosen provider. If generation fails, try a different provider with `-provider qwen` or `-provider google`.

0.14 Section 7. Protocol Suite (v0.3.0)

0.14.1 SYNTHONIC_PERTURBATION

Controlled perturbation of the 10-primitive tuple to identify load-bearing vs. decorative primitives. Full spec: `SYNTHONIC_PERTURBATION.md`.

Python API

```

from synthonicon import PerturbationEngine
from synthonicon.registry import global_catalog

dimer = global_catalog.get("carboxylic_acid_dimer")
engine = PerturbationEngine()

# Full primitive Jacobian --- all primitives +/-1 tier
jac = engine.sweep_all(dimer, delta_g=-12.0)
print(f"Baseline \xi_CP: {jac.baseline_xi_CP:.3f} nats")
print(f"Most sensitive: {jac.most_sensitive.primitive_name}")

for pr in sorted(jac.results, key=lambda x: abs(x.delta_xi_CP),
                 reverse=True):
    print(f"    {pr.primitive_name:<20}    {pr.delta_xi_CP:+.4f} nats    [{pr.sensitivity}]")

# Fault injection --- single points of failure
fault = engine.fault_injection(dimer, delta_g=-12.0)
print(f"Robust: {fault['system_robust']}")
print(f"Single points of failure: {fault['single_points_of_failure']}")

# Path to target \xi_CP
path = engine.find_path_to_target(dimer, delta_g=-12.0,
                                  target_xi_CP=7.5, optimize_primitives=["F", "K"])
print(f"Target reached: {path['target_reached']}")

```

CLI

```

# Full Jacobian sweep
syncon perturb sweep carboxylic_acid_dimer --delta-g -12.0

# JSON output
syncon perturb sweep carboxylic_acid_dimer --delta-g -12.0 --format json

# Fault injection
syncon perturb fault-injection carboxylic_acid_dimer --delta-g -12.0

# Path to target
syncon perturb pathfind carboxylic_acid_dimer --delta-g -12.0 --target 7.5 --optimize F,K

```

Sensitivity labels: CRITICAL ($\Delta\xi_{CP} \geq 3.0$ nats), HIGH (≥ 1.5), MEDIUM (≥ 0.5), LOW (< 0.5).

0.14.2 SYNTHONIC_TRAJECTORY

Encode D_∞ systems as ordered step sequences; validate Axiom 6 (temporal grounding) and kinetic accessibility. Full spec: SYNTHONIC_TRAJECTORY.md.

Python API

```
from synthonicon import TemporalSynthonAgent, Synthon
from synthonicon.models import (
    Dimensionality, Topology, RecognitionMode, Polarity,
    Fidelity, KineticCharacter, Granularity,
    InteractionGrammar,
)

agent = TemporalSynthonAgent("proline_aldol_cycle")

# Build step synthons (all must have D_inf or R_ for Axiom 4)
base = dict(
    dimensionality=Dimensionality.TEMPORAL,
    topology=Topology.CYCLIC_BOWTIE,
    recognition_mode=RecognitionMode.DYNAMIC_CATALYTIC,
    polarity=Polarity.DONOR_ACCEPTOR,
    granularity=Granularity.MESOSCALE,
    interaction_grammar=InteractionGrammar.SELECTIVE_SEQ,
)

enamine = Synthon(name="enamine_formation", fidelity=
    Fidelity.HIGH, kinetic_character=KineticCharacter.FAST,
    **base)
ts = Synthon(name="c_c_bond_form", fidelity=
    Fidelity.MEDIUM, kinetic_character=KineticCharacter.
    MODERATE, **base)
hydrol = Synthon(name="hydrolysis_reset", fidelity=
    Fidelity.HIGH, kinetic_character=KineticCharacter.FAST,
    **base)

agent.add_step(enamine, "enamine_formation", delta_g=-15.0)
agent.add_step(ts, "c_c_bond_form",
    delta_g_ddagger=97.0) # rate-determining
agent.add_step(hydrol, "hydrolysis_reset", delta_g=-25.0,
    is_reset=True)

result = agent.validate_all()
print(f"Valid: {result.overall_valid}")
print(f"Axiom 6 satisfied: {result.axiom6_satisfied}")
print(f"Reset step: {result.reset_step}")
print(f"Kinetic traps: {result.kinetic_traps}")
print(f"Full cycle candidacy: {result.full_cycle_candidacy:.3f
    }")
```

CLI

```
# Validate a named set of steps (must exist in catalog)
```

```
syncon trajectory validate --steps enamine_formation,
    c_c_bond_form,hydrolysis_reset --reset hydrolysis_reset

# Criticality scan per step
syncon trajectory criticality --steps enamine_formation,
    c_c_bond_form,hydrolysis_reset --varma-probe
```

Key checks:

- Axiom 4 (per step): must have D_{inf} or R₋ (dynamic catalytic)
- Axiom 6 (cycle): reset step returns system to S_{t0} (keyword or is_reset=True)
- K_{trap} / ΔG > 100 kJ/mol: flagged in kinetic_traps

0.14.3 SYNTHONIC_ENSEMBLER

Multi-synthon composition: pairwise compatibility, emergent properties, system ξ_{CP}. Full spec: SYNTHONIC_ENSEMBLER.md.

Python API

```
from synthonicon import EnsembleCatalog

ensemble = EnsembleCatalog()
ensemble.add("carboxylic_acid_dimer")      # by catalog name
ensemble.add("proline_aldol_cycle")
ensemble.add("nitroso_radical_cucurbituril_anion_rotaxane_synthon")

# Pairwise compatibility + emergent properties
report = ensemble.check_pairwise()
print(f"Consistent: {report.is_consistent}")
print(f"Score: {report.consistency_score:.0%}")

for entry in report.pairwise_matrix:
    print(f"    {entry.component_a} <-> {entry.component_b}: {entry.result}")

for ep in report.emergent_properties:
    print(f"    {ep.property_name}: detected={ep.detected}")

for axiom, status in report.axiom_propagation.items():
    print(f"    {axiom}: {status}")

# System-level ξCP with interface overhead
result = ensemble.compute_system_xi_CP(delta_g_assembly=-85.0,
    interface_overhead_bits=1.5)
print(f"\xi_CP system: {result['xi_CP_system_nats']:.4f} nats
    [{result['efficiency_tier']}]")
```

CLI

```

# Pairwise matrix + emergent properties
syncon ensemble check --components carboxylic_acid_dimer,
    proline_aldol_cycle

# Criticality probe across ensemble
syncon ensemble probe --criticality --components
    carboxylic_acid_dimer,proline_aldol_cycle

# System thermodynamics
syncon ensemble thermo --components carboxylic_acid_dimer,
    proline_aldol_cycle --delta-g-assembly -85.0

```

Emergent properties detected:

- Emergent Criticality -- ensemble degeneracy_strength > individual average + 0.10 and ≥ 0.40
- Granularity Amplification ($G_- \rightarrow G_+$) -- ≥ 3 G_- components with Axiom 3 superlinear induction
- Interface Fidelity Degradation -- incompatible pair(s) with mean F > 0.5

Interface overhead rule: each additional bit of overhead adds $\ln(2) \approx 0.693$ nats to ξ_{CP} via Landauer identity.

0.14.4 SYNTHONIC_RETRODESIGN

Constraint-directed retrosynthetic decomposition: recursive axiom-pruned tree.
Full spec: SYNTHONIC_RETRODESIGN.md.

Python API

```

from synthomnicon import RetrodesignEngine

engine = RetrodesignEngine()

# Decompose by catalog name
tree = engine.decompose(
    "carboxylic_acid_dimer",
    max_depth=3,
    prune_axioms=[1, 2, 4, 6],
)

print(f"Valid decompositions : {len(tree.valid_leaves)}")
print(f"Pruned branches      : {tree.pruned_count}")
print(f"Valid synthon set      : {tree.valid_synthon_set}")

# Decompose by notation string (9-primitive \langle...\rangle
# format)
target = SynthonNotation.from_string(
    "\langle{D_triangle, D_infinity}; T_cage; R_superset+
    ddagger; "

```



```

    "P_pm; F_eth; K_mod; G_gimel; Gamma_and(SELECTIVE);
    Phi_sub\angle"
)
# or pass the string directly
tree2 = engine.decompose(notation_str, max_depth=2)

# Walk the full tree
def walk(node, depth=0):
    status = "PRUNED" if node.is_pruned else ("LEAF" if node.
    is_leaf else "node")
    print(" " * depth + f"[{status}] {node.branch_name}")
    for v in node.violations:
        print(" " * depth + f"    Axiom {v.axiom}: {v.
        condition}")
    for child in node.children:
        walk(child, depth + 1)

walk(tree.root)

```

CLI

```

# Decompose by catalog name
syncon retrodesign carboxylic_acid_dimer

# Custom depth and axiom set
syncon retrodesign carboxylic_acid_dimer --max-depth 4 --prune
    -axioms 1,4,6

# Decompose by notation string
syncon retrodesign "\langle{D_triangle, D_infinity}; T_cage;
    R_superset; P_pm; F_eth; K_mod; G_gimel; Gamma_and(
    SELECTIVE); Phi_sub\angle"

# JSON output
syncon retrodesign carboxylic_acid_dimer --format json

```

Pruning rules:

Axiom	Name	Condition
1	Fidelity Floor	$T_+ + P_{+/-} + F_{\ell} \rightarrow \text{PRUNE}$
2	Propagation Barrier	sub-tuple claims G_+ but lacks Γ_{or} or $T_{\text{network}} \rightarrow$ PRUNE
4	Grammar Mismatch	$\Gamma_- \rightarrow$ without D_{inf} or $R_- \rightarrow$ PRUNE
6	Grounding Fail	D_{inf} without reset mechanism $\rightarrow \text{FLAG}$

D	D_molecular	D_molecular	D_molecular
T	T_cyclic	T_cyclic	T_cyclic
R	R_non_covalent	R_non_covalent	R_non_covalent
F	F_hbar	F_eth	F_eth v
K	K_mod	K_fast	K_fast v (wait)
G	G_gimel	G_beth	G_beth v
Phi	Phi_sub	Phi_sub	Phi_sub

Result: $\langle D_{\text{and}}; T; R_{\text{supseteq}}; P_{\dots}; F_{\text{eth}}; K_{\text{fast}}; G; \Gamma_{\dots}; \Phi_{\text{sub}}; S \rangle$

CONFLICT primitives are shown in red. A meet with no conflicts means the two synthons are mutually substitutable within the partial order.

Python API:

```
from synthonicon.algebra import meet
result = meet(s1, s2)
print(result.to_notation())      # unified notation
print(result.conflicts)          # list of conflicted primitive
                                # names
print(result.is_valid)           # True if no CONFLICT sentinels
```

0.15.2 syncon join -- Lattice Join (Least Upper Bound)

```
syncon join SYNTHON_A SYNTHON_B [--format text|json]
```

Componentwise maximum: for ordered primitives takes the higher value; for categoricals, shared value or CONFLICT. The result is the "minimal synthon that subsumes both" -- the minimal ceiling of the design space spanned by the two inputs.

Key use case: join is how to find the minimal upgrade that satisfies both of two design targets. If $\text{join}(s1, s2)$ has no CONFLICTs, there exists a valid synthon above both.

Criticality rule: Φ_c dominates in both meet and join. If either input has Φ_c , the output has Φ_c . This encodes the physical fact that criticality is not a property that can be averaged away.

Python API:

```
from synthonicon.algebra import join
result = join(s1, s2)
```

0.15.3 syncon path -- Valid-Swap Path Search

```
syncon path SOURCE DESTINATION [--max-hops 5] [--xi-tolerance 2.0]
```

BFS over the directed valid-swap graph. Edges exist where HotSwapEngine returns APPROVED or CONDITIONAL. Restricted to the same $\{D, T\}$ cluster (cross-domain edges always fail the HotSwap D/T exact-match requirement). Accumulates $|\Delta_{CP}^{\xi}|$ along the path; terminates if the cumulative cost exceeds -xi-tolerance.

Important: The path graph is **asymmetric**. `path(A -> B)` may succeed while `path(B -> A)` fails. This happens whenever B has lower F than A -- the F-floor constraint blocks downgrade edges. This is correct behavior: a swap is not the same operation as its reverse.

Example:

```
# This succeeds (1 hop, Delta\xi = +0.312 nat):
syncon path
  Dithiadiazolyl_Phthalocyanine_Columnar_Stacking_Synthon \
    nitroso_radical_redox_synthon_pair

# This fails (F downgrade blocked):
syncon path nitroso_radical_redox_synthon_pair \

  Dithiadiazolyl_Phthalocyanine_Columnar_Stacking_Synthon
```

Output (success):

```
Path: Dithia -> nitroso
Hop 1: Dithia -> nitroso    Delta\xi = +0.312 nat    [APPROVED]
Total Delta\xi_CP: 0.312 nat    Hops: 1
```

Python API:

```
from synthonicon.algebra import find_path
from synthonicon.registry import global_catalog

path = find_path(s1, s2, catalog=global_catalog, max_hops=5,
  xi_tolerance=2.0)
if path.found:
    for hop in path.hops:
        print(f"  {hop.source.name} -> {hop.dest.name}    Delta\xi = {hop.delta_xi:+.3f}")
```

0.15.4 syncon tensor -- Ensemble Prediction (Tensor Product)

```
syncon tensor SYNTHON_A SYNTHON_B [--lambda 0.5] [--format
text|json]
```

Computes the effective primitive tuple of a two-component assembly. Composition rules:

Primitive	Rule	Physical meaning
D	Set union	Ensemble spans all dimensions of components
T	Max (promote)	Most complex topology governs assembly
R	Promote (dynamic > static)	Dominant interaction governs propagation
P	Donor-acceptor if both present	Mixed polarity creates directional system
F	Min (bottleneck)	Weakest fidelity limits the ensemble
K	Min (bottleneck)	Slowest step governs kinetics
G	Max	Largest-scale control propagates up
Γ	AND-composition	Partner requirements are additive
Φ	Φ _c propagates	Criticality is contagious
S	Product	Combined assembly has product valency

ξcorrection: $\xi_{\text{ens}} = \xi_1 + \xi_2 - \lambda \cdot I(s_1; s_2)$, where $I(s_1; s_2)$ is the mutual information between tuple vectors (shared primitive values), and λ is the overlap discount (default 0.5).

Important consequence: If tensor(s1, s2) drops F to F_0 , a subsequent criticality_lift on the result will be BLOCKED -- the F floor is correctly enforced without any explicit guard in the downstream pipeline.

Python API:

```
from synthonicon.algebra import tensor
result = tensor(s1, s2, lambda_=0.5)
print(result.to_notation())           # effective tuple
print(result.xi_ensemble)             # effective \xi_CP
print(result.overlap_discount)        # lambda . I(s;s) correction
```

0.15.5 syncon lift -- Natural Transformation (Cross-Domain Migration)

```
syncon lift SYNTHON_NAME TARGET [--format text|json]
```

where TARGET is one of: temporal, spatial, critical, molecular.

Applies a primitive-rewriting map that migrates the synthon to a new domain while preserving as much structure as possible. Output shows a side-by-side diff of changed primitives.

Available lifts:

Target	Primitive changes	Guard condition
temporal	$D_{\wedge} \rightarrow D_{\infty}$, $R \rightarrow R_{_}$, $K_{\text{fast}} \rightarrow K_{\text{mod}}$, $\Gamma \rightarrow \Gamma_{\rightarrow}(\text{SEQUENTIAL})$	None
spatial	$D_{\wedge} \rightarrow D_{_}$, $T \rightarrow T_{_}$ if not spatial, $G_{\rightarrow} \rightarrow G_{_}$	None
critical	$\Phi_{\text{sub}} \rightarrow \Phi_{\text{c}}$, $G \rightarrow G_{_}$	$F \geq F_{\text{hrequired}}$ (Axiom 5)
molecular	$D_{\infty} \rightarrow D_{\wedge}$, $R_{\rightarrow} \rightarrow R_{\subseteq}$, $\Gamma_{\rightarrow} \rightarrow \Gamma_{\wedge}(\text{SELECTIVE})$	None (forgetful)

Criticality lift guard (Axiom 5 enforcement): If $F < F_{\text{h}}$, the lift is BLOCKED with explanation. A synthon must have high thermodynamic fidelity to carry a reliable criticality signature.

Example:

```
syncon lift
  Dithiadiazolyl_Phthalocyanine_Columnar_Stacking_Synthon
critical
```

Output:

```
Lift: critical --- Dithia
Changed primitives:
  Phi: Phi_sub -> Phi_c
  G: G_gimel -> G_aleph
Result: \langle D_{\wedge}; T_{\rightarrow}; R_{\supseteq}; P_{+}; F_{\text{hbar}}; K_{\text{mod}}; G_{\rightarrow}; \Gamma_{\wedge}(\text{SPECIFIC}); \Phi_{\text{c}}; 1:1 \rangle
Note: Axiom 6 injection: temporal lift requires closed-cycle grounding before assembly.
```

Python API:

```
from synthonicon.algebra import criticality_lift,
    lift_to_temporal, lift_to_spatial, project_to_molecular
```

```

result = criticality_lift(s)
if result.blocked:
    print(f"BLOCKED: {result.block_reason}")
else:
    print(result.result_synthon.to_notation())
    for warning in result.warnings:
        print(f"Warning: {warning}")

```

0.15.6 syncon pipeline -- Composable Design Pipeline

```
syncon pipeline START_SYNTHON --step STEP [--step STEP ...]
```

A composable pipeline that chains algebra operations with automatic ξ -CP threading and fail-fast logging. Implements a Writer+Maybe monad: each step carries the accumulated cost (Writer) and continues from the last valid state even on BLOCKED steps (soft Maybe).

Step syntax:

```
op:argument[:key=value[:key=value...]]
```

Step	Syntax	Operation
meet	meet:synthon_name	Lattice meet with named synthon
join	join:synthon_name	Lattice join with named synthon
tensor	tensor:synthon_name or tensor:synthon_name:lambda	Tensor product
lift	lift:temporal / lift:spatial / lift:critical / lift:molecular	Natural transformation
path	path:target_name or path:target_name:xi_tolerance	Path search to target

Full example:

```

syncon pipeline
  Dithiadiazolyl_Phthalocyanine_Columnar_Stacking_Synthon \
  --step join:nitroso_radical_redox_synthon_pair \
  --step lift:critical \
  --step path:Varma_synthon:xi_tolerance=1.5

```

Output:

```

Pipeline trace:
Start:  Dithia      \langle...; F_\hbar; K_mod; G_; ...;
       Phi_sub; 1:1\rangle
-----
Step 1: join(nitroso)
       F: F_eth -> F_hbar (max)   K: K_fast -> K_mod   G: G_beth
       -> G_gimel
       Delta\xi_CP = +0.519 nat      [PASS]
Step 2: lift(critical)
       Phi: Phi_sub -> Phi_c   G: G_gimel -> G_aleph
       F = F_hbar >= F_hbar   eligibility gate passed
       Delta\xi_CP = +0.000 nat      [PASS]
Step 3: path(Varma_synthon, tol=1.5)
       Hop 1: ... -> Varma   Delta\xi = +0.847 nat      [APPROVED]
       Delta\xi_CP = +0.847 nat      [PASS]
-----
Final:  \langle...; F_\hbar; K_mod; G_; ...; Phi_c; 1:1\rangle
Total Delta\xi_CP: 1.366 nat   Steps: 3   Blocked: 0

```

Python API (direct monadic chaining):

```

from synthonicon.algebra import DesignPipeline
from synthonicon.registry import global_catalog

nitroso = global_catalog["nitroso_radical_redox_synthon_pair"]
varma    = global_catalog["Varma_synthon"]

result = (
    DesignPipeline
    .start(dithia)
    .join(nitroso)
    .lift("critical")
    .path("Varma_synthon")
    .result()
)

result.print_trace()
print(f"Total Delta\xi_CP: {result.total_delta_xi:.3f} nat")
print(f"Final tuple: {result.current_synthon.to_notation()}")
print(f"Blocked steps: {result.blocked_count}")

```

Pipeline semantics:

- **Writer effect:** `total_delta_xi` accumulates across all steps.
- **Maybe effect:** **BLOCKED** steps are logged but do not terminate the pipeline. The pipeline continues from the last successfully transformed synthon.
- **Fail condition:** A step that raises an exception (e.g., synthon not found, BFS graph unreachable) terminates the pipeline with an **ERROR** status distinct

from BLOCKED.

Behavioral verification:

1. **F-bottleneck propagation:** `tensor(A, B)` dropping F to F_{eth} $\rightarrow$ subsequent `lift:critical` is BLOCKED without any explicit guard needed.
2. **Path asymmetry:** `path:nitroso` from Dithia succeeds (F upgrade direction); `path:Dithia` from nitroso fails (F downgrade blocked).
3. **Join-enables-path:** `join(nitroso)` upgrades F to F_{hbar} $\rightarrow$ `path:Varma` at $\Delta\xi = +0.847$ nat succeeds where it would have failed from the lower-F starting point.

0.15.7 syncon distance -- Tuple Distance (v0.4.0)

```
syncon distance SYNTHON_A SYNTHON_B [--directed] [--format
text|json]
```

Computes the weighted Hamming-style distance between two synthon tuples in the eleven-primitive space. By default reports the **symmetric** distance $d(A, B) = d(B, A)$. With `-directed`, also reports the asymmetric directed distances $d(A \rightarrow B)$ and $d(B \rightarrow A)$ and the asymmetry index.

Primitive weights (default):

Primitive	Weight	Notes
D	1.5	Dimensionality — highest weight (structural backbone)
T	1.2	Topology
R	1.0	Reactivity
P	0.8	Polarity
F	1.0	Fidelity (with asymmetric F-floor penalty)
K	0.6	Kinetics
G	0.9	Granularity
Γ	0.7	Grammar
Φ	1.1	Criticality phase
S	0.08	Stoichiometry
Ω	0.7	Topological protection index (when set)

Example:

```
syncon distance photon electron
```

Example output (text):

```

Distance: photon <-> electron
d(sym) = 3.20
d(photon -> electron) = 3.05
d(electron -> photon) = 3.35
Asymmetry = 0.30

Primitive breakdown:
D      0.00    D_inf <-> D_inf           [match]
T      1.20    T_x <-> T_|             [mismatch]
R      0.00    R_irr <-> R_irr          [match]
P      0.80    P_gamma <-> P_e           [mismatch]
F      0.00    F_\hbar <-> F_\hbar       [match]
K      0.00    K_fast <-> K_fast         [match]
G      0.90    G_ <-> G_                 [match]
\Gamma  0.00    \Gamma_\otimes <-> \Gamma_\otimes
[match]
Phi     0.00    Phi_sub <-> Phi_sub       [match]
S      0.30    None <-> None              [match]
\Omega  ---      (not set on either)

```

JSON output (-format json):

```

{
  "synthon_a": "photon",
  "synthon_b": "electron",
  "d_sym": 3.20,
  "d_a_to_b": 3.05,
  "d_b_to_a": 3.35,
  "asymmetry": 0.30,
  "primitives": {
    "T": {"weight": 1.2, "match": false},
    "P": {"weight": 0.8, "match": false}
  }
}

```

Python API:

```

from synthonicon.algebra import tuple_distance

d = tuple_distance(synthon_a, synthon_b)
print(f"Distance: {d:.3f}")

```

Quantum particle distance results (v0.4.0 playground):

0.15.8 Combined Workflow Example

```

# Step 0: Measure primitive-space separation

```

Pair	d(sym)	Dominant mismatch
proton $\leftrightarrow$ electron	1.50	P (charge sign)
spin $\leftrightarrow$ qubit	2.10	T + K (spatial $\leftrightarrow$ linear; trap $\leftrightarrow$ slow)
photon $\leftrightarrow$ qubit	3.80	T + P + K
proton $\leftrightarrow$ spin	6.00	Max separation (T + P + K + G stacking)

```

syncon distance Dithia nitroso --directed    # d(sym)=2.10;
        asymmetry=0.40

# Step 1: Check whether two synthons are lattice-compatible
syncon meet Dithia nitroso                  # identifies F/K/G gaps
syncon join Dithia nitroso                  # identifies minimal common
        ceiling

# Step 2: Can we swap directly?
syncon hotswap Dithia nitroso               # HotSwap check
syncon path Dithia nitroso                 # finds 1-hop path, Delta\
        xi = +0.312 nat

# Step 3: What does the ensemble look like?
syncon tensor Dithia nitroso               # effective tuple (F->min->
        F_eth, K->min->K_mod)

# Step 4: Can we lift the joined ensemble to critical?
syncon pipeline Dithia \
    --step join:nitroso \
    --step lift:critical \
    --step path:Varma:xi_tolerance=1.5
# -> join restores F_hbar -> criticality lift passes -> path
    to Varma succeeds

```

0.15.9 DesignPipeline Python API

```

from synthonicon.algebra import DesignPipeline,
    global_catalog

# Load starting synthon
start = global_catalog.get("proline_aldol_cycle")

# Build a multi-stage pipeline
pipeline = (
    DesignPipeline()
    .start(start)
    .meet("amide_dimer")                # lattice meet

```

```

    .join("carboxylic_acid_dimer")      # lattice join
    .tensor("nitroso_radical_redox_synthon_pair", lambda_=0.5)
    .lift("critical")                   # natural transformation
    .path("soai_pyrimidyl_autocatalytic_cycle", xi_tolerance
    =2.0)
)

# Execute and get results
result = pipeline.result()

# Inspect results
print(f"Final synthon: {result.current_synthon.name}")
print(f"Total Delta\xi_CP: {result.total_delta_xi:+.3f} nat")
print(f"Steps: {len(result.steps)}, Blocked: {result.
    blocked_count}")

# Print full trace
result.print_trace()

```

0.16 Design Scripts (.syn DSL) -- Complete Reference

Version: 0.4.1 (Phase 3a -- Monadic Foundation · Molecular Domain Catalog)
 Total Scripts: 20 (Batch 1: 01-05 + corrected; Batch 2: 06-16) Success
 Rate: 18/20 (90%) -- 2 intentional F-floor pedagogical demonstrations (01,
 04)

0.16.1 Quick Reference Table

0.16.2 Running Design Scripts

```

# Run a single design (text output)
syncon run designs/03_retrosynthetic_meet_join.syn

# Run with JSON output and save
syncon run designs/14_complex_strategy_composition.syn --
    format json --save result.json

# Dry-run (parse + validate without executing)
syncon run designs/06_catalytic_cycle_optimization.syn --dry-
    run

# Override output format
syncon run designs/10_hierarchical_ensemble.syn --format json

```

0.16.3 Featured Designs (Deep Dive)

Design 03: Retrosynthetic Meet/Join Goal: Demonstrate lattice operations
 on H-bonded dimers

Result: 5/5 steps, $\Delta\xi_{CP} = 0.000$ nat

Script	Status	$\Delta\xi_{\text{CP}}$ (nat)	Key Demonstration
Batch 1			
01_soai_criticality	Blocked (pedagogical)	0.000	F-floor gate: proline(F_eth) cannot lift to Φ_c
01b_soai_criticalit	SUCCESS	0.000	Factor 7 fires; Φ_c lift; meet preserves Φ_c
02_tensor_ensemble_	Partial	13.624	Tensor fidelity bottleneck
02b_tensor_ensemble	SUCCESS	13.624	Tensor $\rightarrow$ join $\rightarrow$ lift
03_retrosynthetic_m	SUCCESS	0.000	meet(F_h, F_eth) $\rightarrow$ F_eth; join $\rightarrow$ F_h
04_hybrid_spatial_t	Blocked (pedagogical)	16.802	Tensor succeeds; F_eth bottleneck blocks Φ_c lift
04b_hybrid_spatial_	SUCCESS	15.425	D $_{\infty}$ hybrid; Axiom 6 pass; lift
05_varma_probe_vali	Partial	0.000	Varma probe, path limits
05b_varma_probe_cor	SUCCESS	0.000	Varma + idempotent join
Batch 2			
06_catalytic_cycle_	SUCCESS	0.962	1-hop path + lift; total cost < 5.0 nat
07_supramolecular_c	SUCCESS	27.518	Sequential tensor $\times 2$
08_retrodesign_sear	SUCCESS	0.000	T_ cluster navigation
09_cross_domain_anz	SUCCESS	0.000	Supramolecular analogy
10_hierarchical_ens	SUCCESS	13.016	tensor(CH, CH); MI discount 7.100 nat
11_criticality_ense	SUCCESS	0.000	Full Varma + Axiom 6
12_thermodynamic_ef	SUCCESS	12.577	crown(F_eth) $\otimes$ CB $\rightarrow$ F_eth; MI 5.698 nat
13_multi_hop_path_c	SUCCESS	0.000	Extended path search
14_complex_strategy	SUCCESS	0.000	lift $\rightarrow$ BLOCKED; mplus $\rightarrow$ join(F_h) $\rightarrow$ lift Φ_c
15_dna_base_pair_la	SUCCESS	0.000	Biological H-bond systems
16_mof_inspired_net	SUCCESS	13.741	cryptand $\otimes$ CB[n] $\rightarrow$ T_cage, F_h; MI

```

1. [PASS]   meet(amide_dimer)   F: F_hbar   F_eth -> F_eth
2. [ASSERT_PASS]   topology == T_bowtie
3. [ASSERT_PASS]   fidelity == F_eth
4. [PASS]   join(carboxylic_acid_dimer)   F: F_eth   F_hbar ->
   F_hbar
5. [ASSERT_PASS]   fidelity == F_hbar

```

Key Insights:

- Meet/join form a lattice on ordered primitives (F, K, G)
- Categorical primitives (D, T, R, P, Γ) require exact match
- Zero thermodynamic cost (abstract algebraic transformations)

Design 10: Hierarchical Ensemble (Highest MI Discount) Goal: Multi-stage assembly with anion-cation pairing

Result: 7/7 steps, $\Delta\xi_{CP} = 12.424$ nat, MI discount = 8.283 nat

```

1. [PASS]   join(carbonyl)   F->F_eth
2. [PASS]   path(methyl_anion)   1 hop
3. [PASS]   tensor(methyl_cation, lambda=0.6)   Delta\xi=+12.424
   nat   discount=8.283
4. [ASSERT_PASS]   topology == T_linear
5. [ASSERT_PASS]   fidelity == F_eth

```

Key Insights:

- Highest observed MI discount (8.283 nat)
- Complementary charges maximize mutual information
- Without MI: cost would be 20.7 nat (actual: 12.4 nat)

Design 14: Complex Strategy Composition (Nested Or-Patterns) Goal: Three-level fallback chain for robust criticality ascent

Result: 9/9 steps, $\Delta\xi_{CP} = 0.362$ nat

```

1. [BLOCKED]   lift(critical)   F-floor not met
2. [PASS]   mplus(fallback)   switching to fallback
3. [BLOCKED]   join(redox_pair)   CONFLICT on R, P, \Gamma
4. [PASS]   mplus(fallback)   switching to fallback
5. [PASS]   path(redox_pair)   Delta\xi=+0.362 nat   1 hop
6. [PASS]   lift(critical)   Phi: None -> Phi_c
7. [ASSERT_PASS]   criticality_phase == Phi_c
8. [ASSERT_PASS]   phi_c_score > 0.3
9. [ASSERT_PASS]   output.assert(steps <= 15)

```

Key Insights:

- Nested or-patterns work flawlessly
- Fallback chain: `lift → join → path`
- Final pathway cost: 0.362 nat (1-hop path + lift)
- Demonstrates robust error recovery in design pipelines

Design 11: Criticality Ensemble Validation **Goal:** Comprehensive Varma probe validation

Result: 8/8 steps, $\Delta\xi_{CP} = 0.000$ nat

```

1. [PASS] lift(critical)  Phi: None -> Phi_c
2. [ASSERT_PASS]  phi_c_score > 0.3
3. [ASSERT_PASS]  criticality_phase == Phi_c
4. [ASSERT_PASS]  axiom6_satisfied
5. [PASS] join(self) idempotent
6. [ASSERT_PASS]  criticality preserved
7. [ASSERT_PASS]  fidelity == F_hbar
8. [ASSERT_PASS]  output.assert(criticality_ok == true)

```

Key Insights:

- All Varma probe predicates validated
- Axiom 6 temporal grounding satisfied
- Idempotent join preserves criticality phase

0.16.4 Thermodynamic Metrics Summary

Operation Type	Typical $\Delta\xi_{CP}$ (nat)	Notes
meet / join	0.000	Abstract lattice ops
lift	0.000	Natural transformation
path (1 hop)	0.000–0.500	Same D/T cluster
path (multi-hop)	0.500–2.000	Cross-cluster
tensor (cage $\otimes$ bowl)	12–16	High cost ensemble
tensor (anion-cation)	12–14	8.3 nat MI discount
tensor (sequential $\times 2$)	25–30	Accumulated cost

Note: Lattice operations and lifts are abstract algebraic transformations (zero cost). Tensor products model physical co-assembly and carry ξ_{CP} predictions with mutual information discounts.

0.16.5 Constraint Enforcement (Floor Gates)**0.16.6 File Index**

Gate	Condition	Blocked If	Evidence
F-floor (criticality)	$F \geq F_hbar$	$F = F_eth$ or F_ell	Designs 01, 04 (pedagogical demos)
D-floor (criticality)	$D = D_\infty$ OR $G \geq G_gimel$	$D = D_triangle$, $G = G_beth$	Design 04 (after tensor bottleneck)
D/T match (path)	Same {D, T} cluster	D or T differs	Designs 08, 09, 13
Axiom 6 (temporal)	Reset block OR flux	D_∞ without grounding	Design 11 validates

```

designs/
|---- Batch 1 (01-05 + corrected)
|   |---- 01_soai_criticality_path.syn
|   |---- 01b_soai_criticality_corrected.syn
|   |---- 02_tensor_ensemble_design.syn
|   |---- 02b_tensor_ensemble_corrected.syn
|   |---- 03_retrosynthetic_meet_join.syn      <- Featured
|   |---- 04_hybrid_spatial_temporal.syn
|   |---- 04b_hybrid_spatial_temporal_corrected.syn
|   |---- 05_varma_probe_validation.syn
|   \---- 05b_varma_probe_corrected.syn
|---- Batch 2 (06-16)
|   |---- 06_catalytic_cycle_optimization.syn
|   |---- 07_supramolecular_cage_assembly.syn
|   |---- 08_retrodesign_search.syn
|   |---- 09_cross_domain_analogy.syn
|   |---- 10_hierarchical_ensemble.syn        <- Highest MI
|   |---- 11_criticality_ensemble_validation.syn
|   |---- 12_thermodynamic_efficiency.syn
|   |---- 13_multi_hop_path_discovery.syn
|   |---- 14_complex_strategy_composition.syn <- Nested or
|   |---- 15_dna_base_pair_lattice.syn
|   \---- 16_mof_inspired_network.syn
\---- DESIGN_SUMMARY.md / DESIGN_SUMMARY_BATCH2.md

results/
\---- [20 JSON result files]

```

0.16.7 Canonical Cross-Domain Demo (TENSOR_OPS_DEMO.py)

TENSOR_OPS_DEMO.py is the definitive worked-example document for all seven algebraic operations. It is structured for readers familiar with tensor mathematics (category theory, functional analysis, lattice algebra) and serves as both a tutorial and a validation suite.

Run:

```

# Run all 7 sections (18 examples total, ~2--3 min)
python TENSOR_OPS_DEMO.py

```



```
# Run a single section
python TENSOR_OPS_DEMO.py --section meet
python TENSOR_OPS_DEMO.py --section join
python TENSOR_OPS_DEMO.py --section tensor
python TENSOR_OPS_DEMO.py --section lift
python TENSOR_OPS_DEMO.py --section path
python TENSOR_OPS_DEMO.py --section pipeline
python TENSOR_OPS_DEMO.py --section decomp
```

Section Map:

Inline synthon definitions: 16 synthons defined directly in the script -- all registered to `global_catalog`. Includes: `Hv1_human_closed`, `Hv1_human_open`, `AtHv1_silent`, `AtHv1_primed`, `PsHv1_constitutive`, `2GBI_inhibitor`, `HIF_inhibitor`, `cooper_pair`, `majorana_fermion`, `TI_surface_state`, `phonon`, `magnon`, `electron`, `hole`, `GNF2_inhibitor`, `imatinib`.

For SYNTHONICON_LANG.md readers: §"Algebraic Operations Reference" in SYNTHONICON_LANG.md contains the formal specification; this demo is the executable companion.

0.16.8 Next Steps (Phase 3b)

1. **Refinement Types:** Move axiom checks to Pydantic v2 validators
 2. **Predicate Expansion:** Add `kinetic_character`, `dimensionality`, `granularity` to `assert grammar`
 3. **Path Search Optimization:** Add caching for repeated probe calls
 4. **Strategy Library:** Build reusable strategy patterns for common design motifs
 5. **Cost Optimization:** Implement heuristic search for minimum-cost pathways
-

0.17 LLM Tool Layer (v0.4.5)

`synthon_tool.py` and `synthon_agent.py` implement the SynthOmnicon LLM tool layer -- a structured interface that lets any tool-calling LLM interact with the live algebra without being able to hallucinate impossible chemistry. Every proposal is immediately rejected with a precise axiom trace if it fails.

0.17.1 syncon tool -- Single-Shot Tool Dispatch

```
syncon tool OPERATION [OPTIONS]
```

Runs one SynthonTool operation and prints the result (text or JSON).

Operations and their required options:

Examples:

```
# Pairwise distance
```

$\mathfrak{g}$	Operation	Key Examples	Category-Theory Translation
1	meet	(1) Hv1 closedopen — Φ_c absorbing; (2) drug scaffold GLB; (3) AtHv1_silentPsHv1 — K_trap isolates	Product / Heyting GLB
2	join	(1) Hv1 cross-species LUB; (2) pharmacophore ceiling; (3) Cooper pairTI surface — Ω_Z survives	Coproduct / F-floor ratchet
3	tensor	(1) electron $\rightarrow$ exciton theorem (T_linear, no promotion); (2) Majorana $\rightarrow$ T_braid special rule; (3) Cooper pair $\rightarrow$ full superconductor	Bifunctor; $\xi_{ens} = \xi + \xi - \lambda I$
4	lift	(1) AtHv1_silent lift(critical) $\rightarrow$ BLOCKED (K_trap gate); (2) molecular $\rightarrow$ temporal (loop-space functor Ω); (3) temporal $\rightarrow$ spatial (classifying space $B(G)$)	Natural transformation; $+2.303 \text{ nat} = \ln 10$
5	path	(1) AtHv1_silent $\rightarrow$ Hv1_h man_open — 3-hop Kleisli geodesic; (2) 2GBI $\rightarrow$ HIF inhibitor scaffold migration; (3) magnon $\rightarrow$ Cooper pair — blocked (D/T gate)	Lawvere metric geodesic
6	pipeline	(1) Hv1 cross-species conservation (hv1_paper_reproduction.syn v6); (2) SynthonM do-notation chain; (3) fallback or-pattern (mplus)	WriterT+StateT+MaybeT Kleisli
7	decomp	(1) cofactor(Cooper pair, electron) $\rightarrow$ K_slow+G_meso+T_66 (2) principal_decomp Birkhoff atoms; (3)	Inverse bifunctor; Birkhoff theorem

Operation	Required options	Description
validate	-name NAME	Full axiom validation on a catalog entry
criticality	-name NAME [-xi-r FLOAT] [-xi-tau FLOAT]	Varma Φ_c probe
path	-src SRC -dst DST [-max-hops INT]	HotSwap path search
analogies	-name NAME [-limit INT]	Nearest catalog neighbors
distance	-a A -b B	Symmetric + directed distances
meet	-a A -b B	Lattice meet (shared primitive floor + CONFLICT levers)
generate	-description TEXT [-name NAME] [-delta-g FLOAT]	Generate + validate from NL

```

syncon tool distance --a allosteric_domain --b active_site
# OK operation=distance
# Distance: 5.400
# d(allosteric_domain->active_site) = 5.400
# d(active_site->allosteric_domain) = 5.400

# Criticality probe with correlation data
syncon tool criticality --name allosteric_domain --xi-r 8.5 --xi-tau 1e10

# Find 3 nearest catalog neighbors
syncon tool analogies --name condensate_liquid --limit 3

# HotSwap path with up to 8 hops
syncon tool path --src condensate_liquid --dst condensate_gel --max-hops 8

# Lattice meet: find shared primitive floor and state-switching levers
syncon tool meet --a condensate_liquid --b condensate_gel

# Generate and axiom-validate from natural language
syncon tool generate --description "bivalent allosteric inhibitor" --name my_inhibitor

# JSON output
syncon tool distance --a allosteric_domain --b active_site --format json

```

0.17.2 syncon design -- Autonomous LLM Design Agent

```
syncon design --goal TEXT [OPTIONS]
```

Runs the autonomous SynthonDesignAgent loop: the LLM proposes a synthon encoding, every proposal is immediately axiom-validated, the Φ_c probe is applied, and the loop continues until convergence criteria are met or -max-iterations is reached.

Options:

Option	Default	Description
-goal TEXT	(required)	Natural-language design goal
-target NAME	None	Catalog entry to require a HotSwap path to
-phi-c-min FLOAT	0.70	Minimum Φ_c score for convergence
-xi-cp-max FLOAT	12.0	Maximum ξ_{CP} (nats) for convergence
-max-iterations INT	10	Maximum LLM refinement iterations
-model TEXT	claude-sonnet-4-6	Claude model ID
-quiet	False	Suppress per-iteration output
-output PATH	None	Save full iteration history as JSON

Examples:

```
# Basic design run
syncon design --goal "bivalent allosteric ABL inhibitor that
  closes T_perp to T_in gap"

# With target path requirement and tighter convergence
syncon design \
  --goal "near-critical condensate scaffold with global
    coordination" \
  --target allosteric_domain \
  --phi-c-min 0.75 \
  --xi-cp-max 10.0 \
  --max-iterations 8

# Quiet mode, save JSON history
syncon design \
  --goal "programmable DNA origami scaffold" \
  --quiet \
  --output design_history.json
```

Convergence output (Rich table):

Iter	Status	Phi_c	\xi_CP	Stop reason
1	iterating	0.62	13.1	
2	iterating	0.68	12.8	
3	CONVERGED	0.73	11.4	Phi_c=0.730 >= 0.70, \xi_CP=11.40 <= 12.0

0.17.3 Python API

```

from synthon_tool import SynthonTool, ToolResponse

# All operations return ToolResponse dataclass
r: ToolResponse = SynthonTool.distance("allosteric_domain", "
    active_site")
print(r.status)          # "ok"
print(r.distance)        # 5.4
print(r.to_json())       # full JSON

# Dispatch by operation name (used by agent)
r = SynthonTool.dispatch("criticality", name="
    allosteric_domain", xi_r=8.5)

# Full agent loop
from synthon_agent import SynthonDesignAgent,
    ConvergenceCriteria

agent = SynthonDesignAgent(
    goal="bivalent allosteric ABL inhibitor",
    criteria=ConvergenceCriteria(phi_c_min=0.70, xi_cp_max
    =12.0),
    model="claude-sonnet-4-6",
    verbose=True,
)
history = agent.run(max_iterations=10)

# Each record in history:
for r in history:
    print(r.iteration, r.phi_c_score, r.xi_cp, r.converged)

```

0.17.4 Using SYNTHON_TOOL_SCHEMA with any LLM API

```

from synthon_tool import SYNTHON_TOOL_SCHEMA, SynthonTool

# Pass to Anthropic messages.create()
import anthropic
client = anthropic.Anthropic()
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    tools=[SYNTHON_TOOL_SCHEMA],

```

```
    messages=[{"role": "user", "content": "Find the nearest  
analog to allosteric_domain"}],  
)  
  
# Dispatch incoming tool_use blocks  
for block in response.content:  
    if block.type == "tool_use":  
        op = block.input.pop("operation")  
        result = SynthonTool.dispatch(op, **block.input)  
        print(result.to_json())
```

The same SYNTHON_TOOL_SCHEMA works with OpenAI-format APIs (pass as tools=[SYNTHON_TOOL_SCHEMA] with type="function" wrapping).

For more detailed information on the underlying theory, refer to QUANTSYNTHONICON.md and SYNTHONICON.md Section VI.